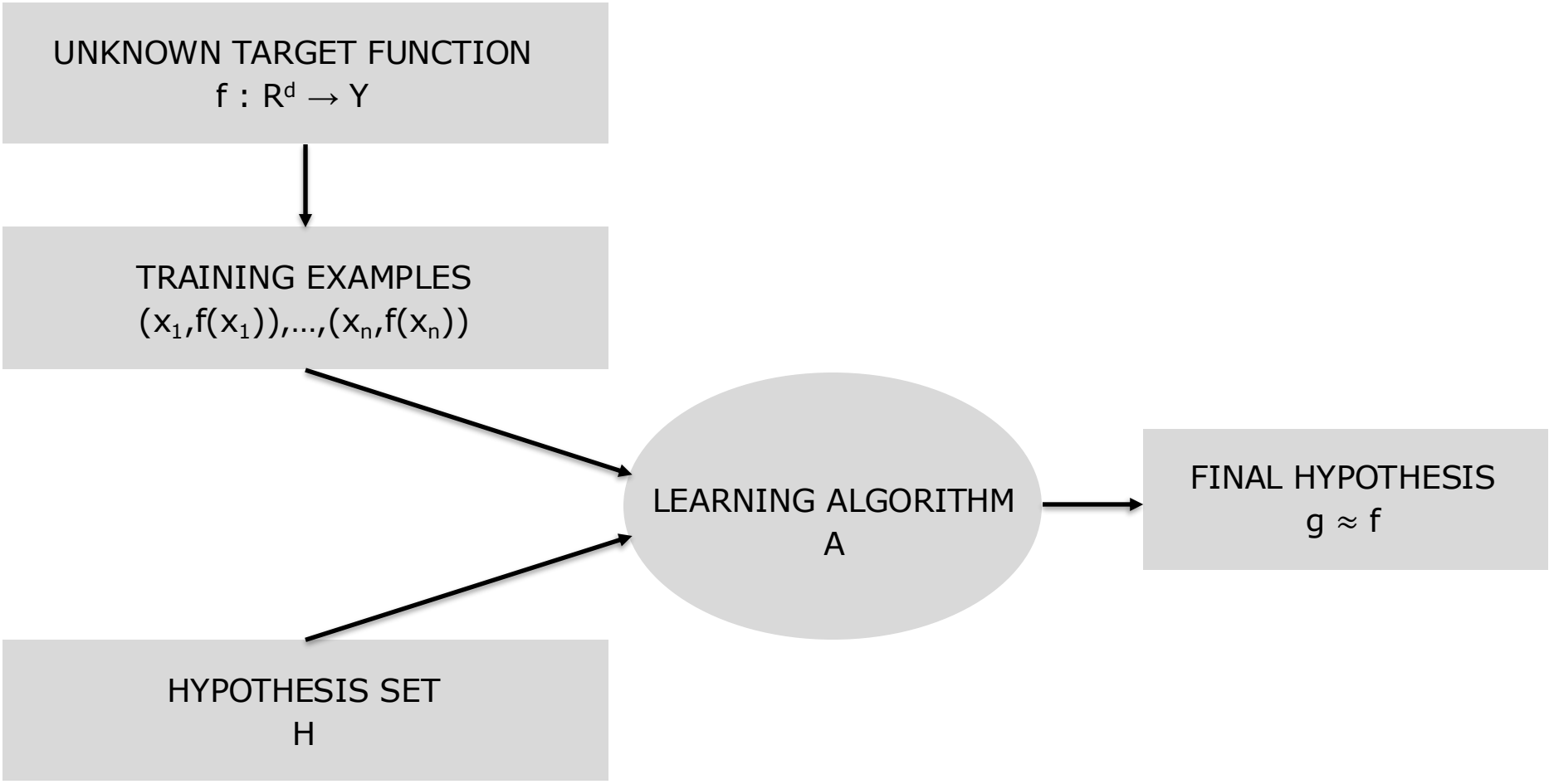
Decision Trees

Supervised Learning



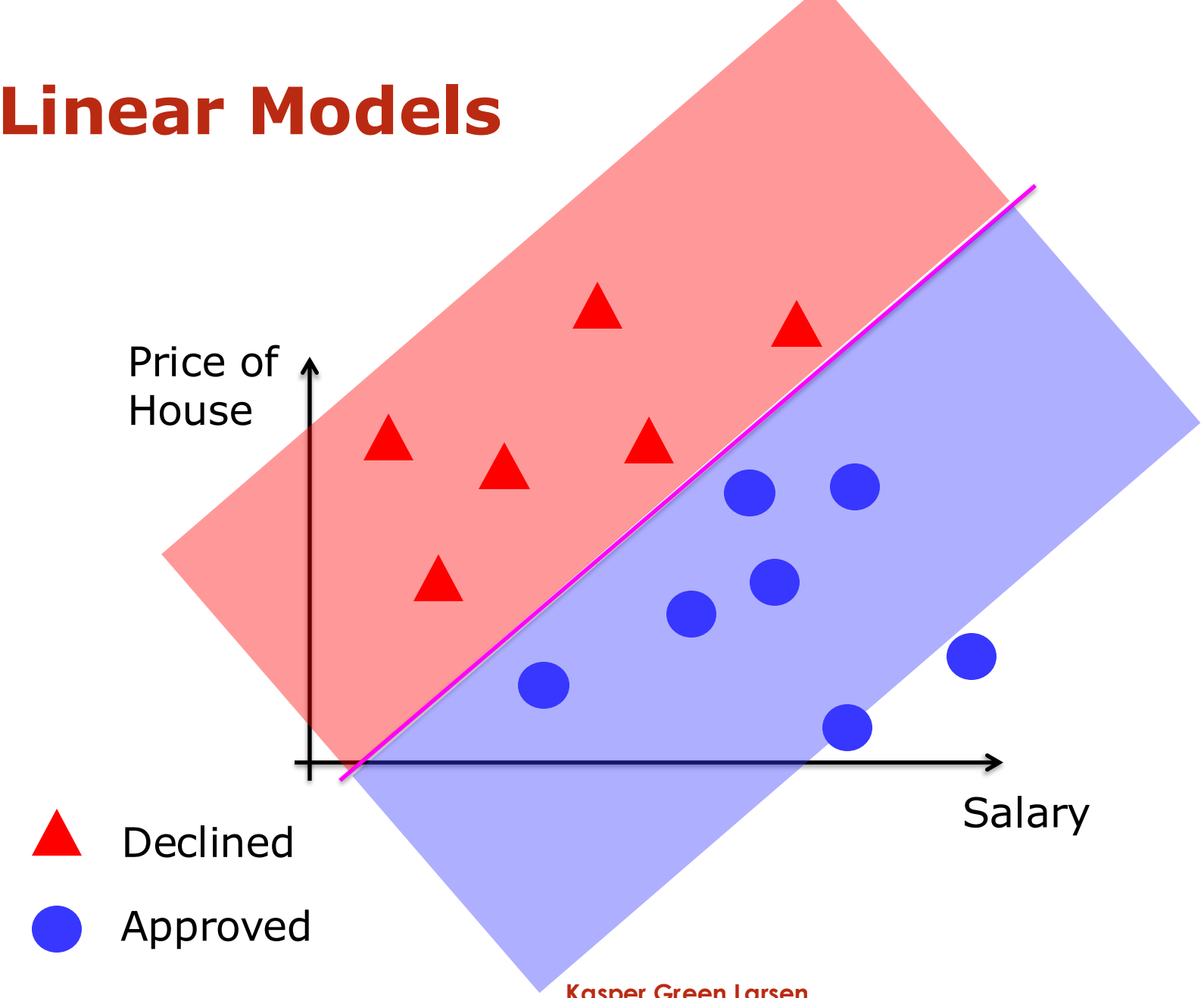
Perceptron Learning Algorithm (PLA)

- Linear Model:
  - Features: $x=(1,x_1,\ldots,x_d)$
  - Labels: $\{-1,1\}$

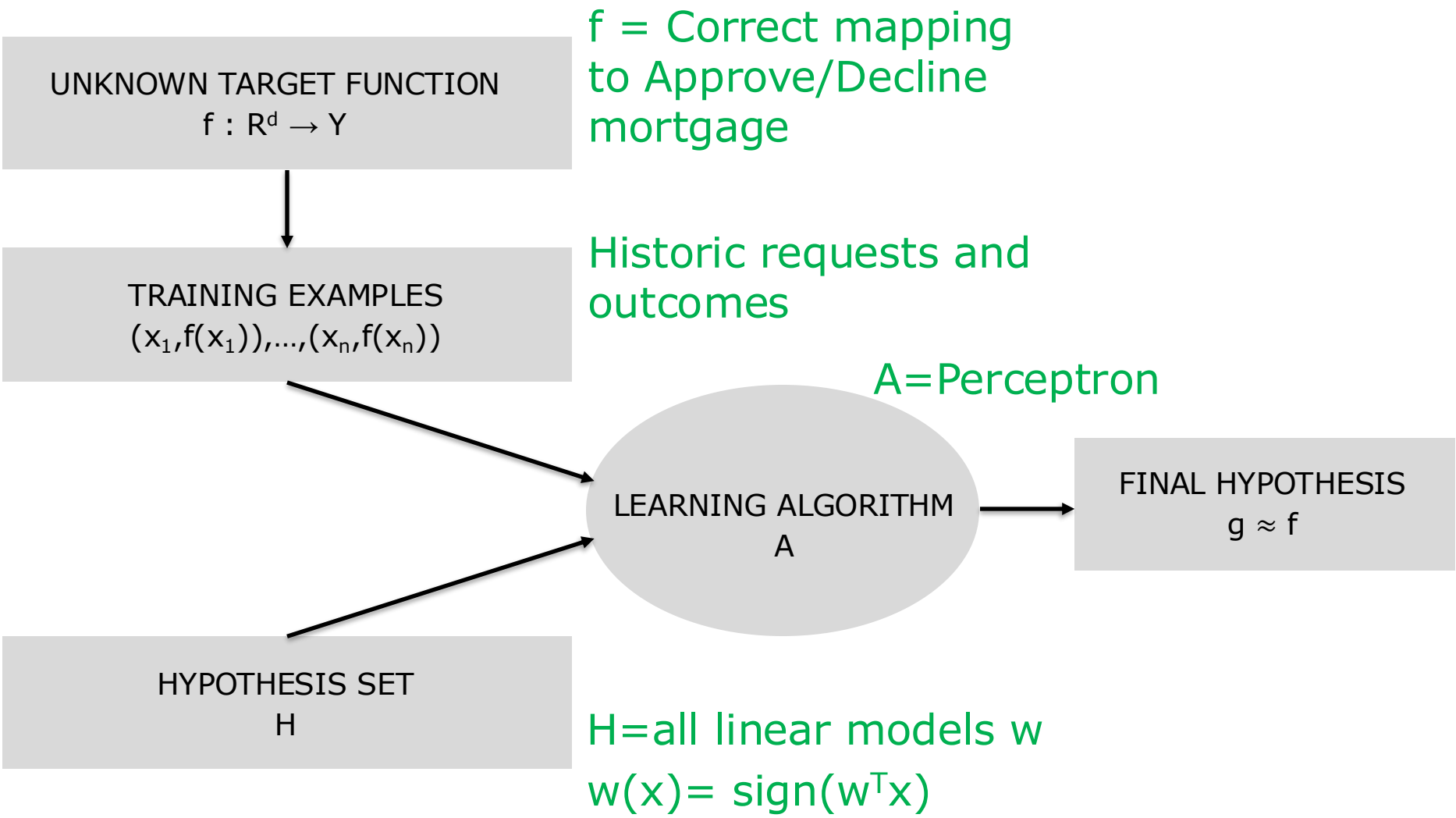
  - Parameters of hypothesis w: $(w_0, w_1,\ldots, w_d)$
  - Predict: $w(x) = \text{sign}(w_0 1 + w_1 x_1 + \ldots + w_d x_d) = \text{sign}(w^T x)$

- Algorithm:
  - $w = (0,0,\ldots,0)$
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - $w = w + yx$
  - Return w

Linear Models

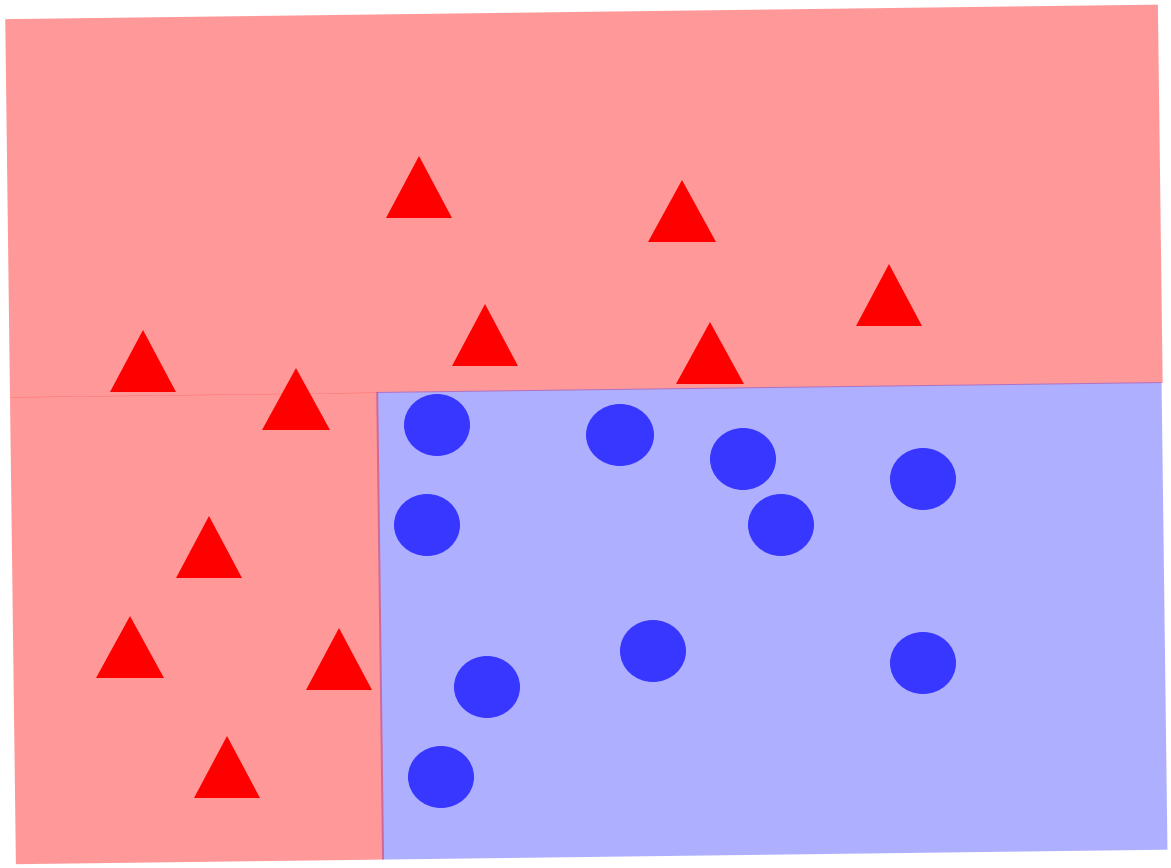


Supervised Learning



Linear Models

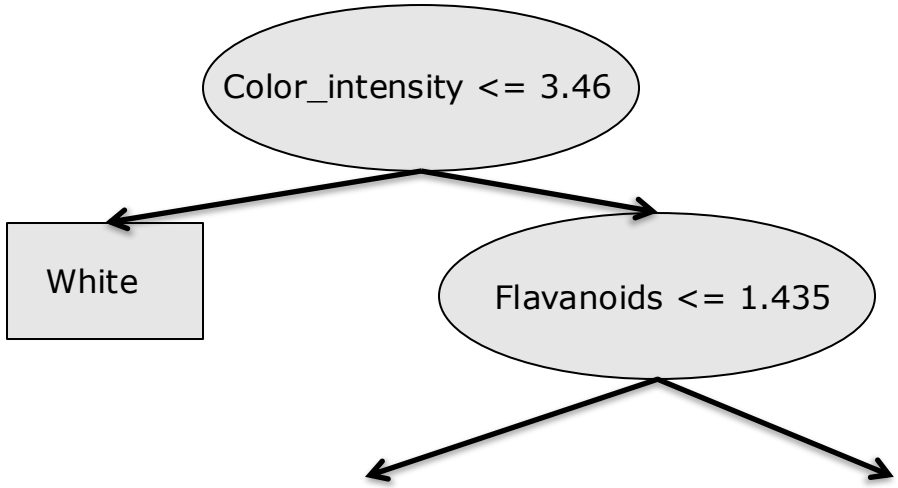
- Shortcomings
  - Data not linearly separable.
  - Other types of hypotheses might be more suitable.



This Lecture



- Decision Trees
  - Different hypothesis set.
  - Different learning algorithms.

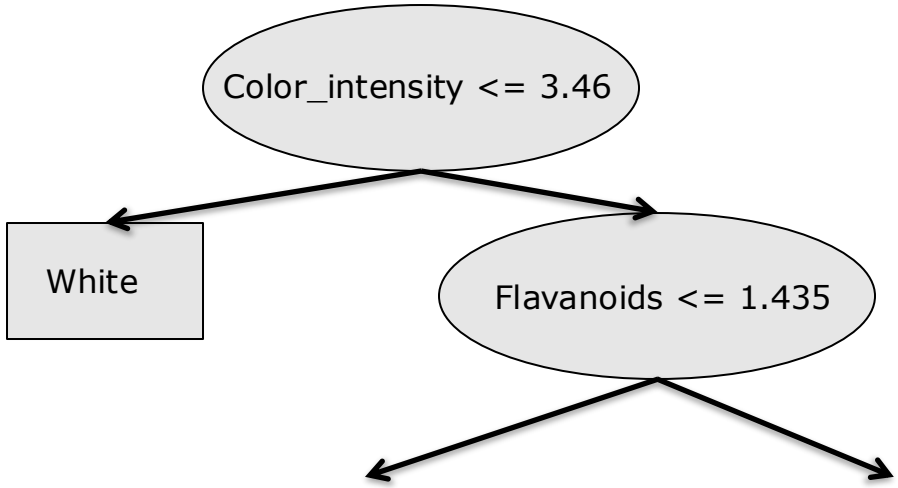


Decision Trees



- Classifying wine.
  - Red, White, Rosé

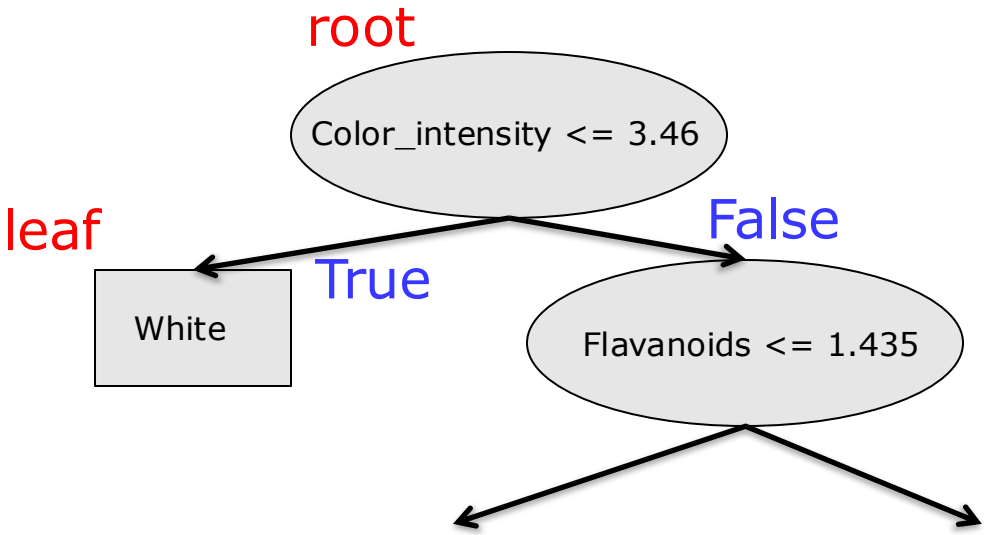
- Predict(x):
  - Start at root node.
  - Test condition in root and descent to relevant child.
  - Upon reach leaf:
    - Return value stored at leaf.



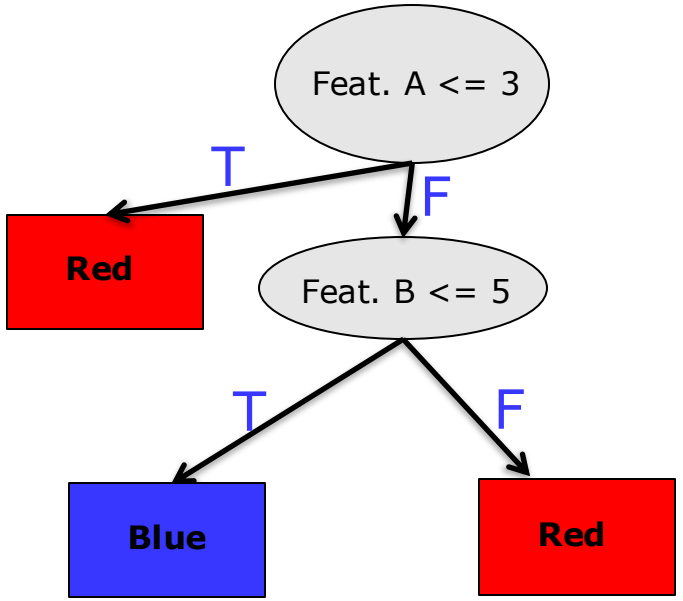
Decision Trees

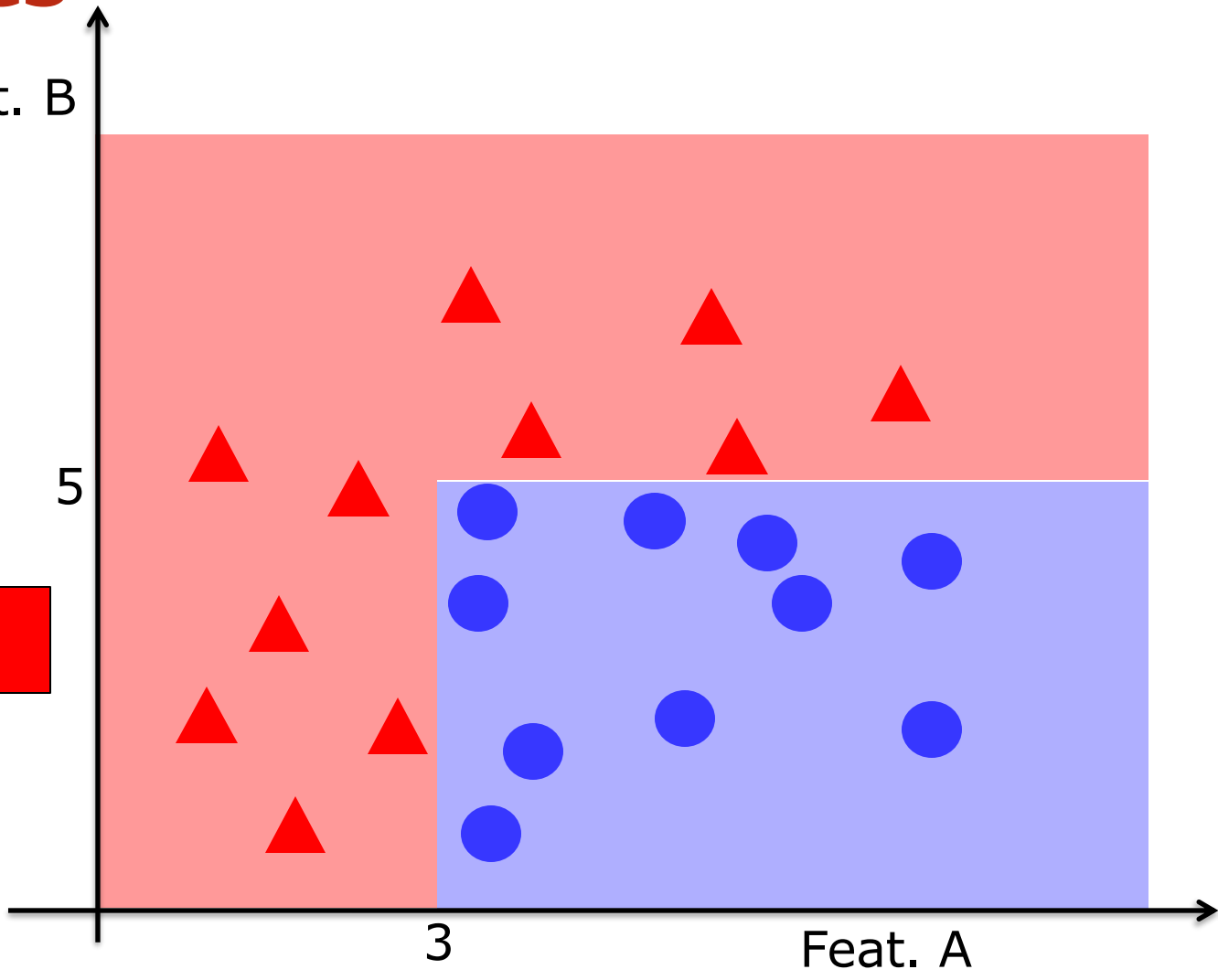


- Classifying wine.
  - Red, White, Rosé
- Predict(x):
  - Start at root node.
  - Test condition in root and descent to relevant child.
  - Upon reach leaf:
    - Return value stored at leaf.
- Node:
  - Question tests **one** feature against **one** value.

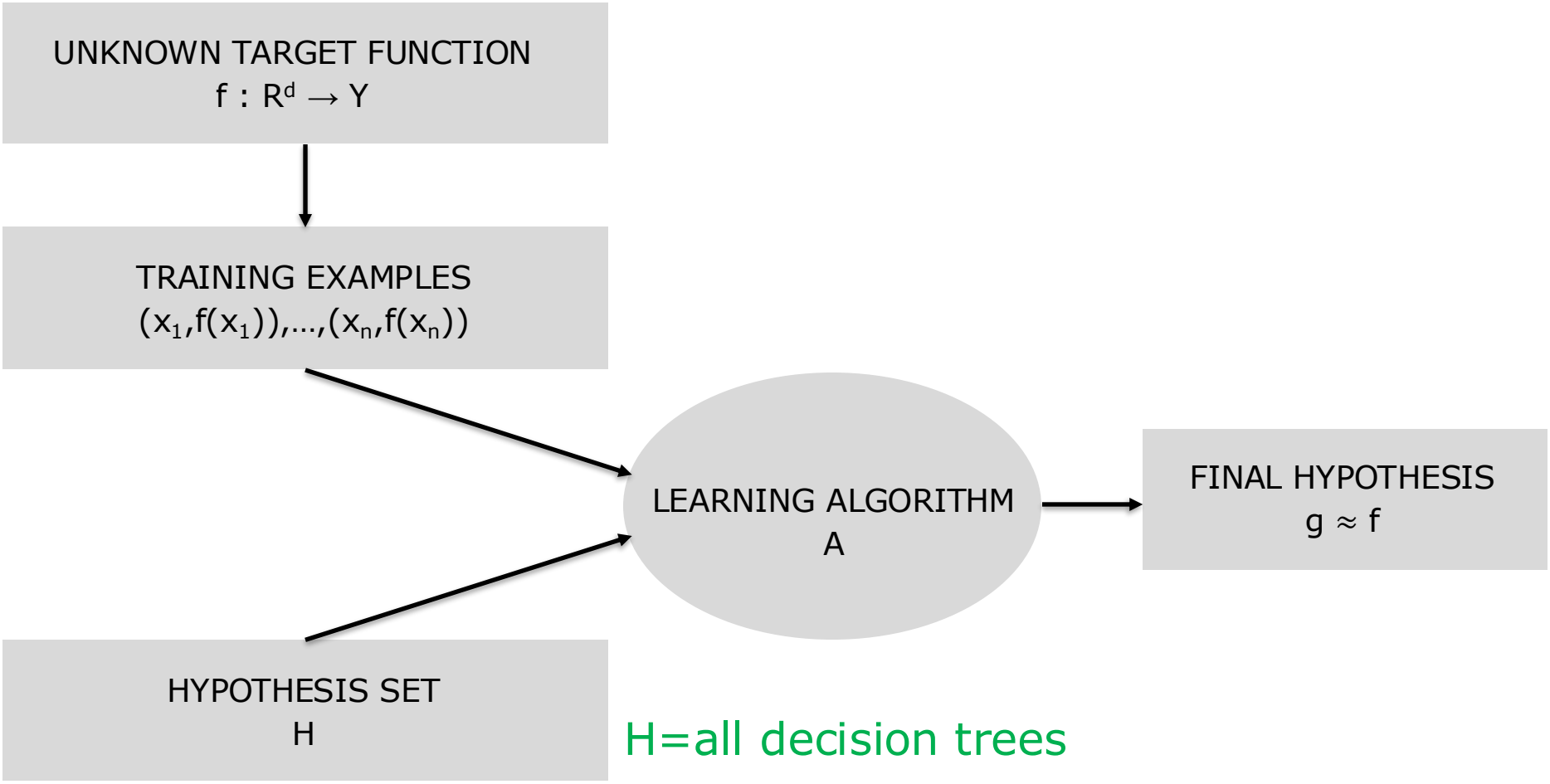


Decision Trees





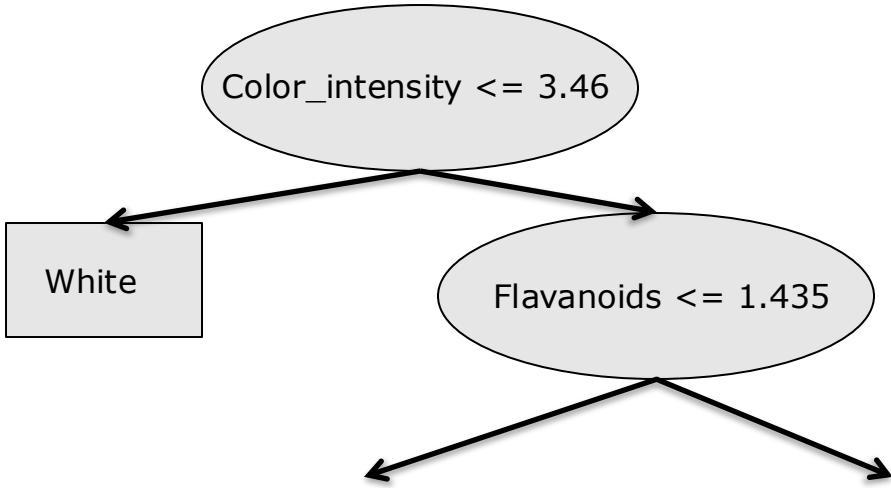
Supervised Learning



Decision Trees

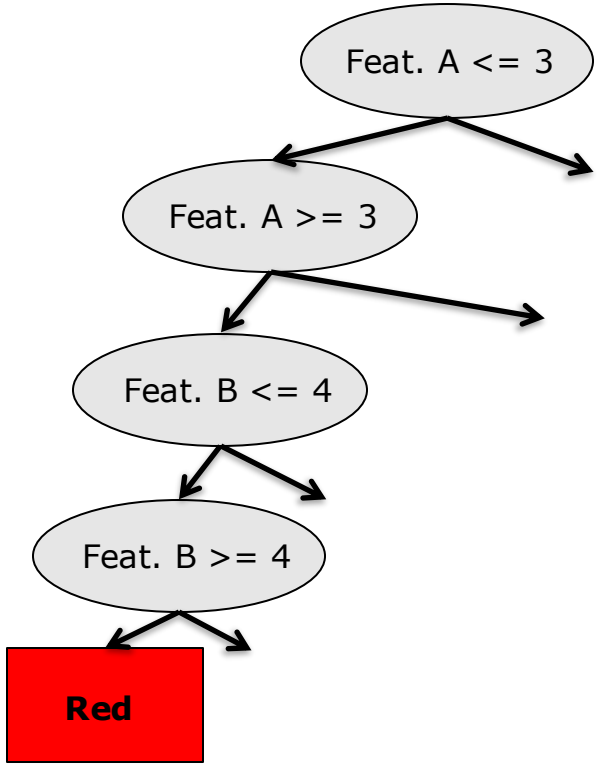


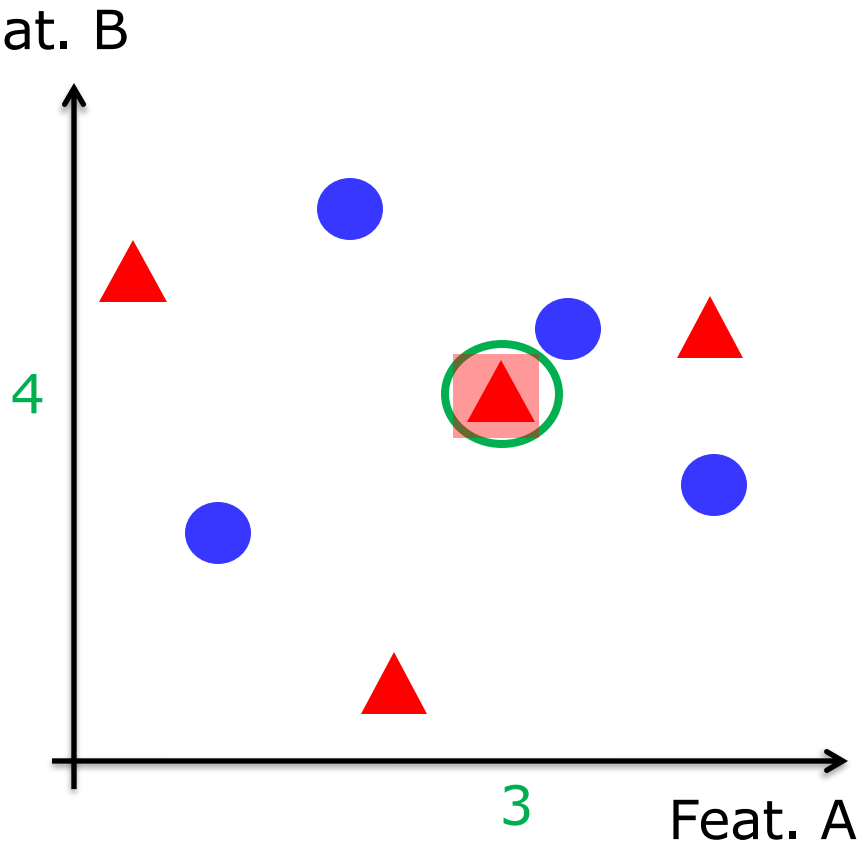
- Learning Algorithm?
  - How do we determine a good decision tree g?
  - Recall: Good if classifies most of training data correctly.
  - Hope: Training data is similar to new unknown data.
    - Good performance on training data hopefully means good performance on new data.
    - $g \approx f$.



Pit Falls

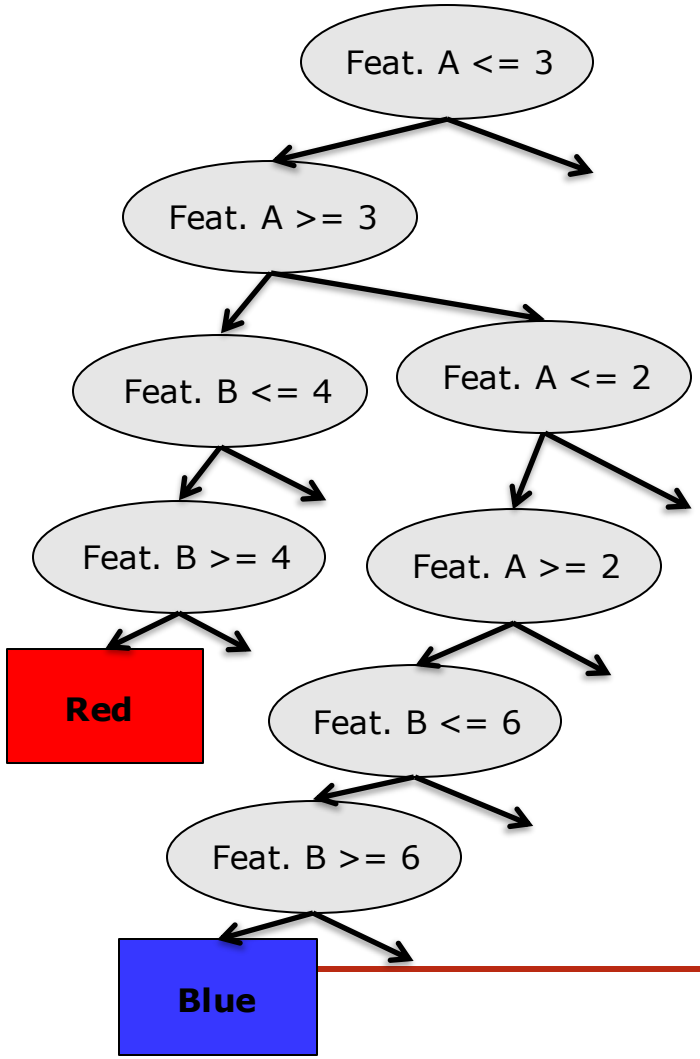
- Always get perfect classification!

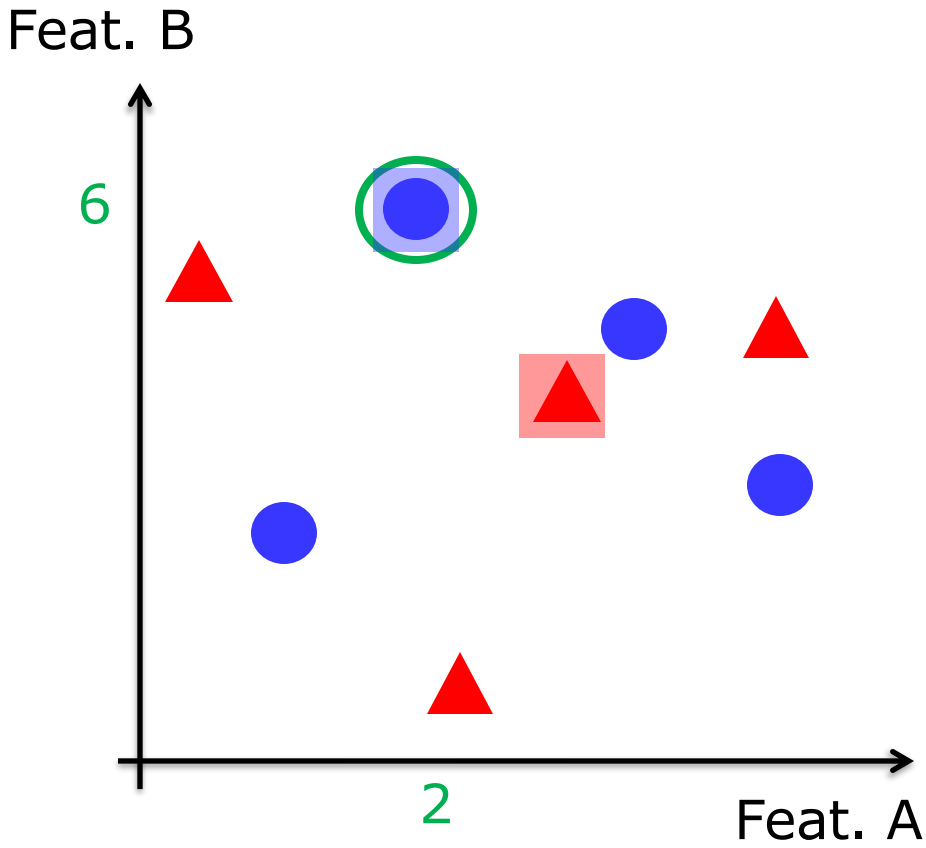




Pit Falls

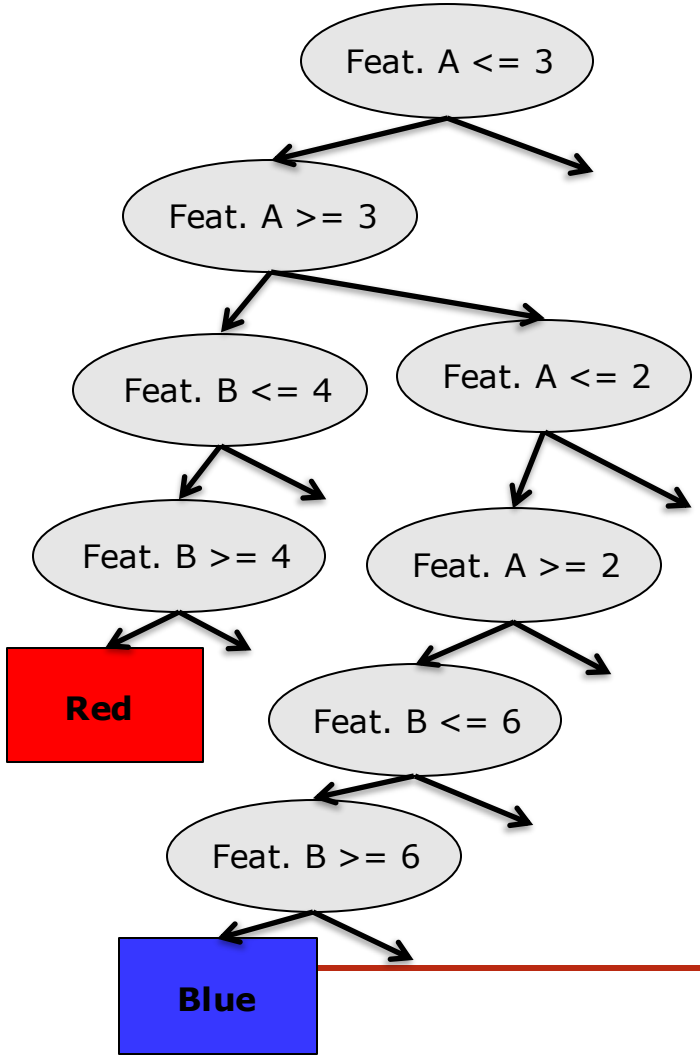
- Always get perfect classification!

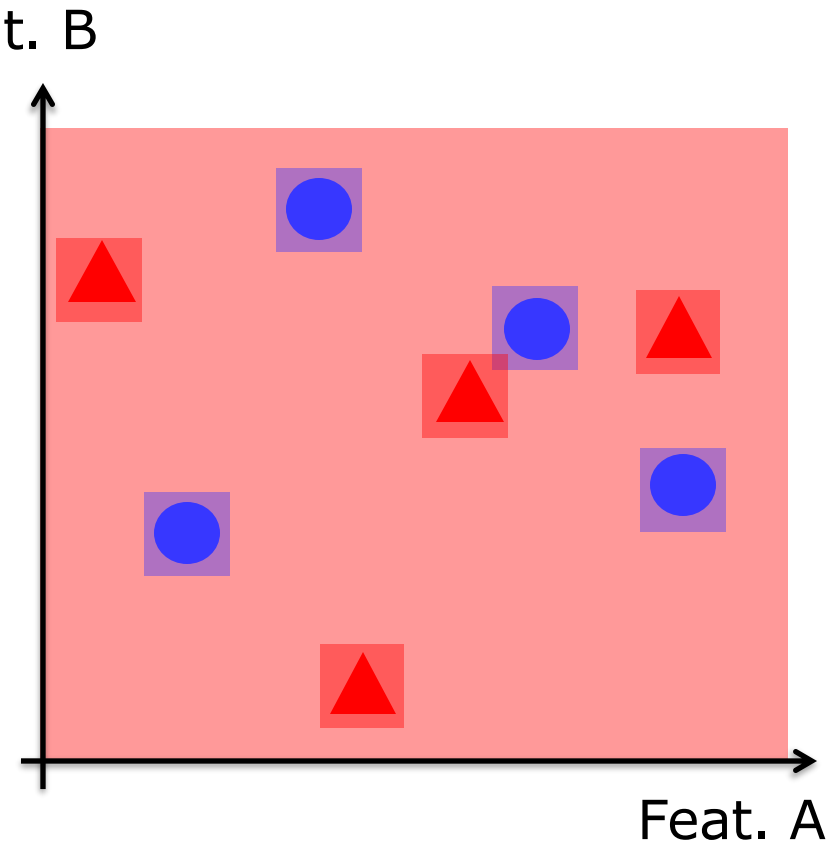




Pit Falls

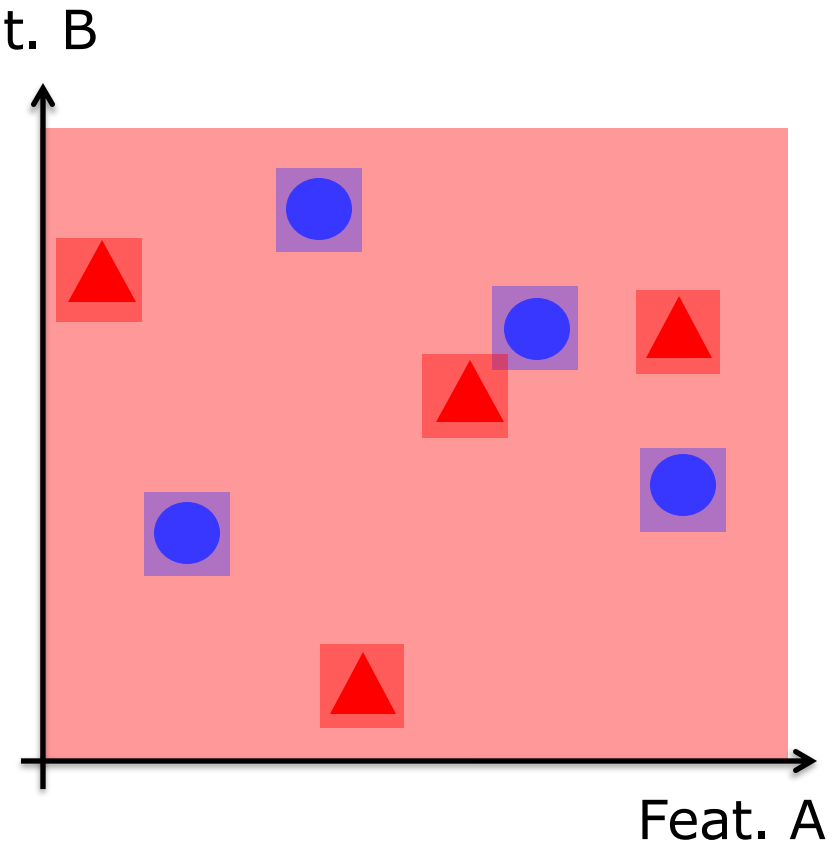
- Always get perfect classification!





Pit Falls

- Always get perfect classification!
  - Seems unlikely that this will have good performance on new unknown data!
  - Somehow extremely complicated decision tree.
  - ***Overfit*** to training data.
    - Will return to this later.

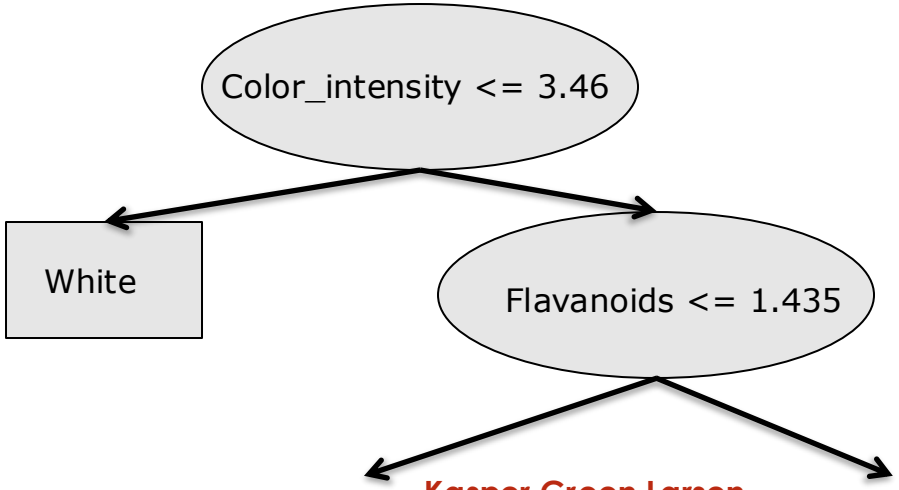


- Idea/Goal:
  - Find *small* decision tree that does well on training data.
  - Prefer simple *explanations* of data. (Will also motivate later).

Small Decision Trees



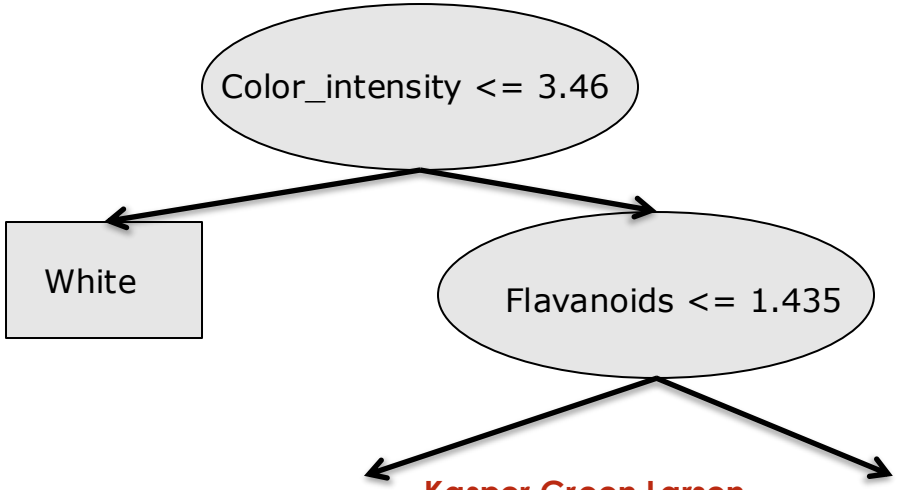
- Learning Algorithm?
  - How do we determine a good decision tree g?
  - Find smallest decision tree perfectly classifying training data?
    - Unfortunately: NP-Hard.
      - If you didn't have course on complexity theory:
        - Probably no efficiently algorithm for finding it.



Small Decision Trees



- Learning Algorithm?
  - How do we determine a good decision tree g?
  - Find best performing decision tree with k nodes?
    - Also NP-hard.

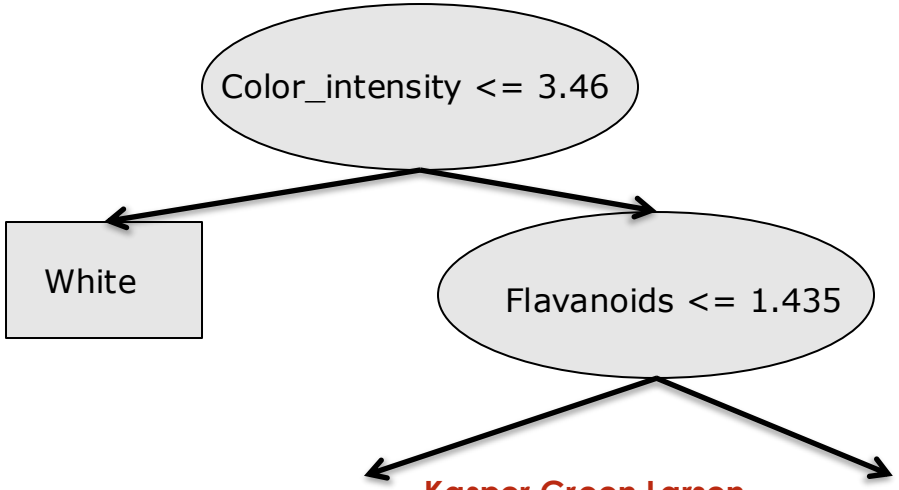


Small Decision Trees



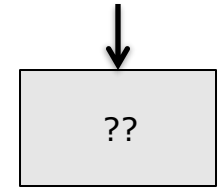


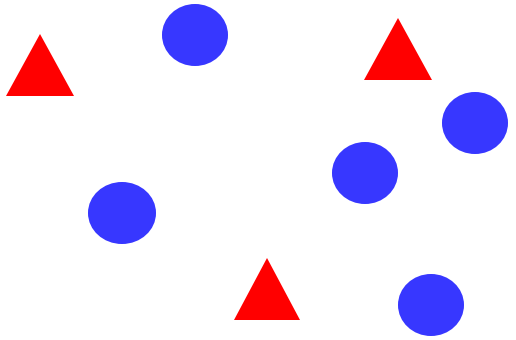
- Learning Algorithm?
  - How do we determine a good decision tree g?
  - We rely on heuristics that hopefully find a good tree.



Small Decision Trees

- Greedy Learning Algorithm.
  - Start in simple case:
    - Our tree may only contain 1 node.
    - Root is a leaf.

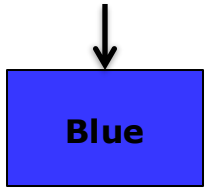


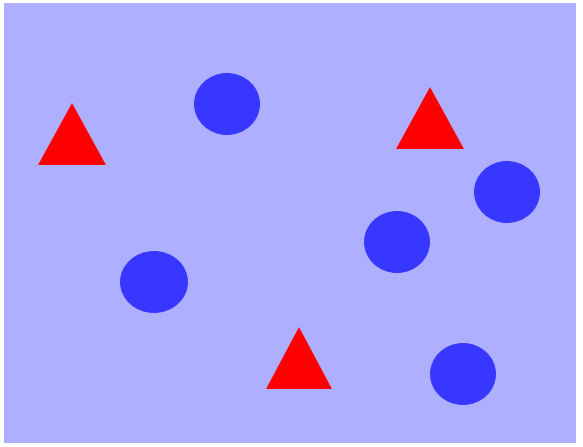


- What is best choice of root?
  - It must return either red or blue.
  - If return red then training accurracy: 3/8
  - If return blue then training accurracy: 5/8
  - The node should return blue.

Small Decision Trees

- Greedy Learning Algorithm.
  - Start in simple case:
    - Our tree may only contain 1 node.
    - Root is a leaf.

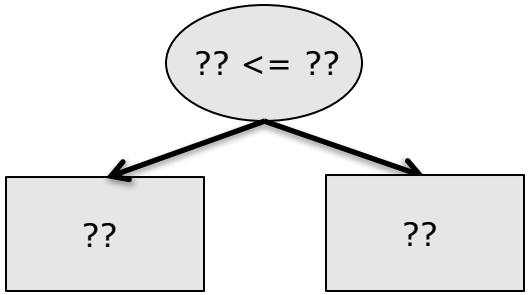


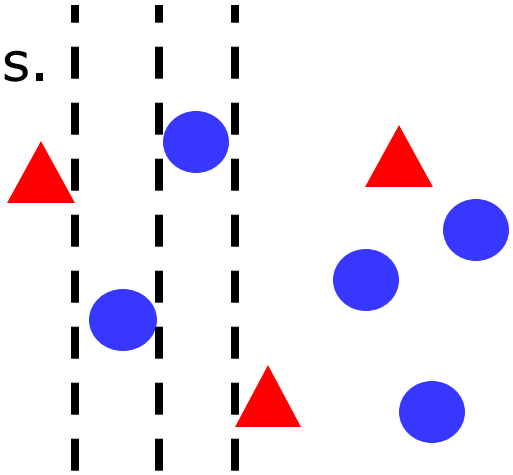


- What is best choice of root?
  - It must return either red or blue.
  - If return red then training accurracy: 3/8
  - If return blue then training accurracy: 5/8
  - The node should return blue.

Small Decision Trees

- Greedy Learning Algorithm.
  - Decision Stumps:
    - One root and two children that are leaves.

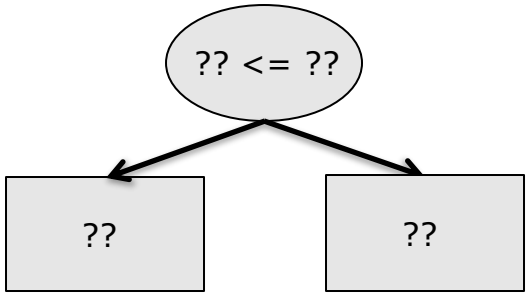


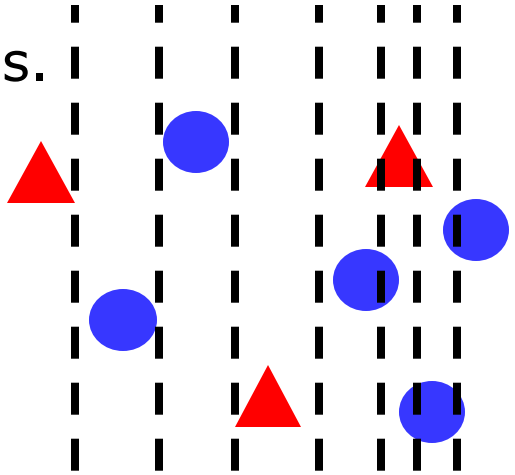


- What is best choice of root?
  - What candidates to consider?
  - For every feature, there are n-1 relevant splits with n training points.
    - Between every pair.

Small Decision Trees

- Greedy Learning Algorithm.
  - Decision Stumps:
    - One root and two children that are leaves.

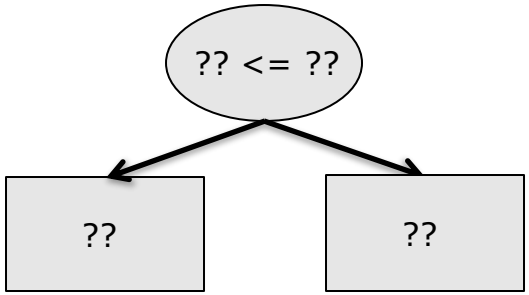


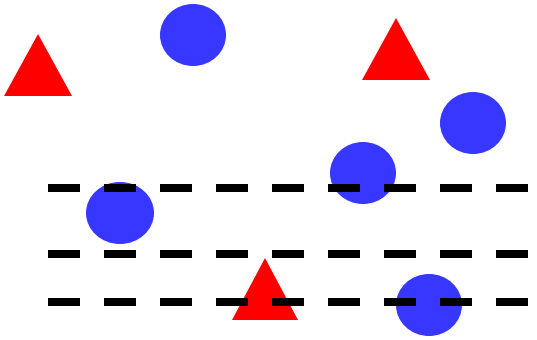


- What is best choice of root?
  - What candidates to consider?
  - For every feature, there are n-1 relevant splits with n training points.
    - Between every pair.

Small Decision Trees

- Greedy Learning Algorithm.
  - Decision Stumps:
    - One root and two children that are leaves.

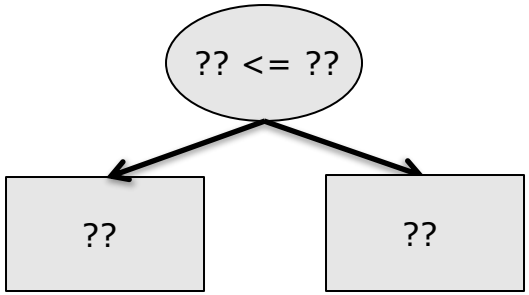


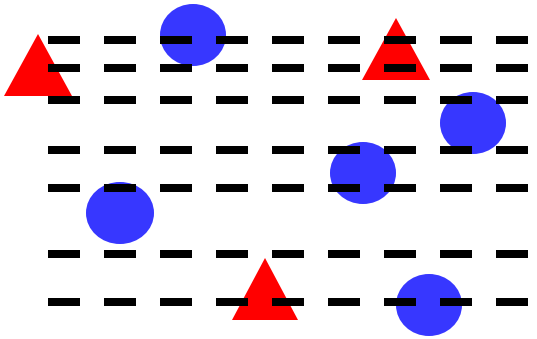


- What is best choice of root?
  - What candidates to consider?
  - For every feature, there are n-1 relevant splits with n training points.
    - Between every pair.

Small Decision Trees

- Greedy Learning Algorithm.
  - Decision Stumps:
    - One root and two children that are leaves.

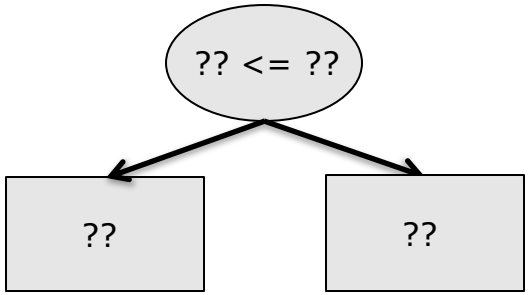


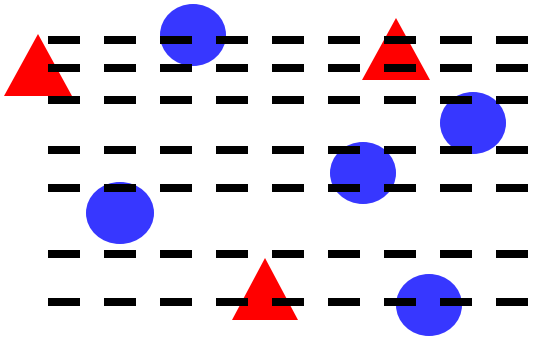


- What is best choice of root?
  - What candidates to consider?
  - For every feature, there are n-1 relevant splits with n training points.
    - Between every pair.

Small Decision Trees

- Greedy Learning Algorithm.
  - Decision Stumps:
    - One root and two children that are leaves.

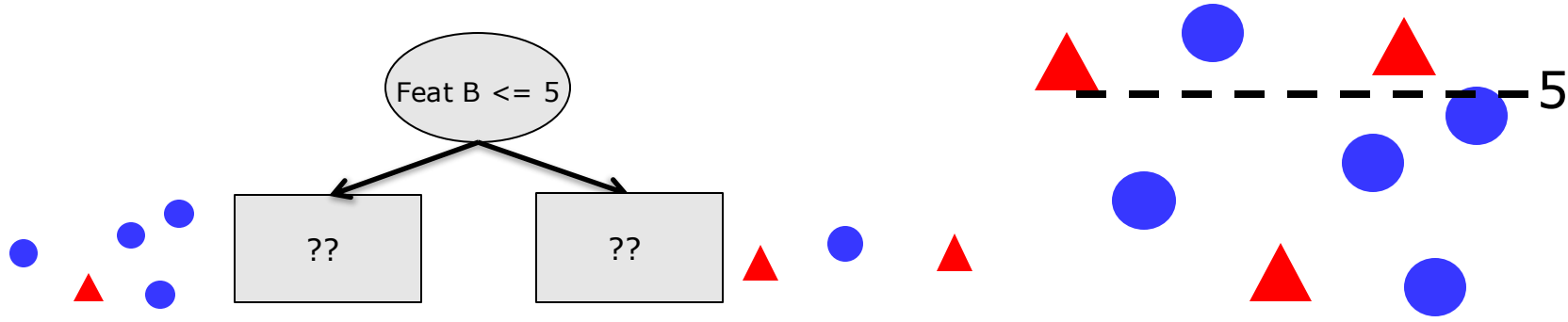




- What is best choice of root?
  - For every feature, there are n-1 relevant splits with n training points.
  - Try all:
    - For each, know how to choose leaves.

Small Decision Trees

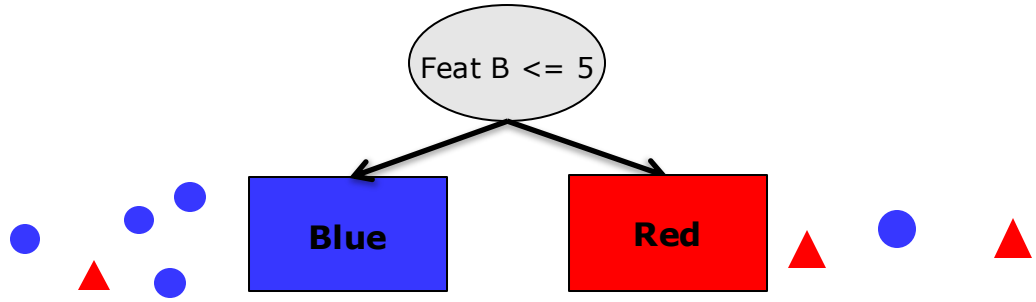
- Greedy Learning Algorithm.
  - Decision Stumps:
    - One root and two children that are leaves.

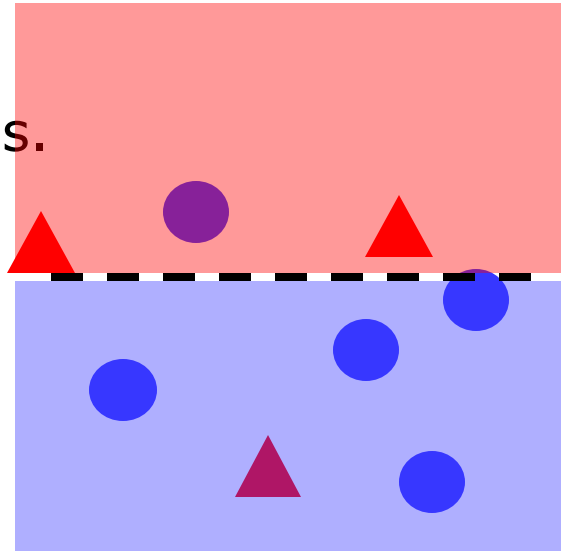


- Testing a split

Small Decision Trees

- Greedy Learning Algorithm.
  - Decision Stumps:
    - One root and two children that are leaves.

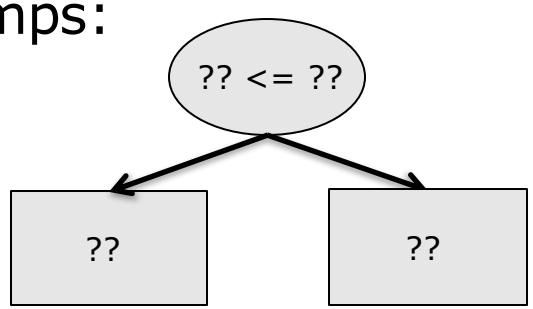


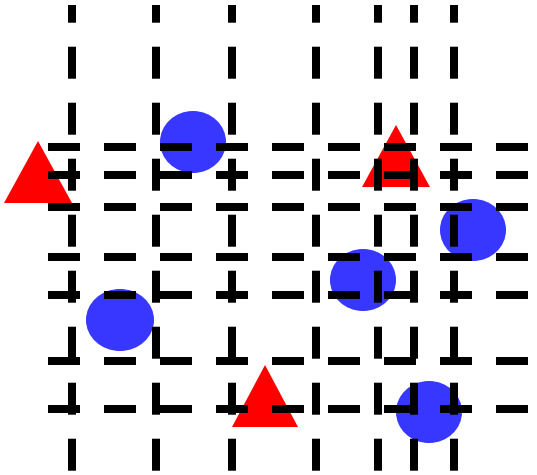


- Accurracy: 6/8

Small Decision Trees

- Greedy Learning Algorithm.
  - Decision Stumps:

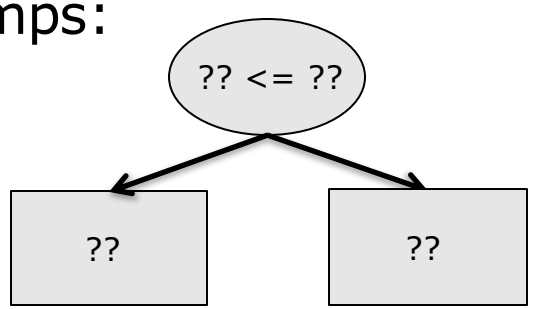


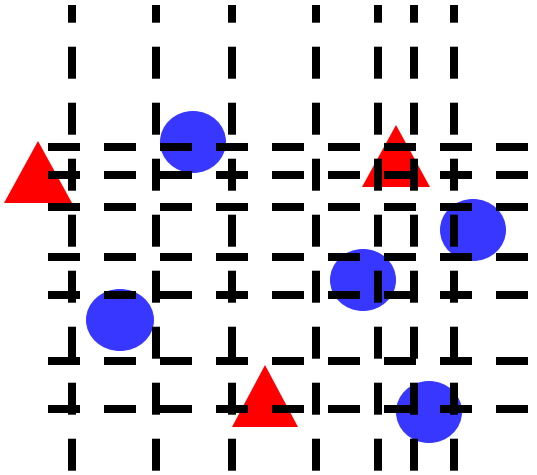


- Find best decision stump:
  - For each of d features:
    - For each of n-1 relevant splits:
      - Try that root, partition points into leaves, assign best leaves, compute accurracy.
  - Use best split found.

Small Decision Trees

- Greedy Learning Algorithm.
  - Decision Stumps:





  - Find best decision stump:

O(d)
- For each of d features:

O(n)
  - For each of n-1 relevant splits:

O(n)
    - Try that root, partition points into leaves, assign best leaves, com[illegible].
  - Use best split four[illegible]

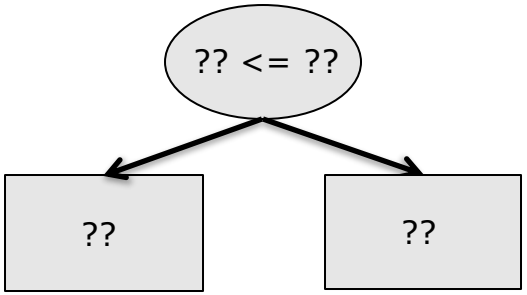
Infeasible on large data sets.

$O(n^2 d)$ running time.

Exercise: Improve to O(n logn d).

Small Decision Trees

- Greedy Learning Algorithm.
  - We can build best stumps quickly.
  - Larger trees?
    - Know NP-hard.

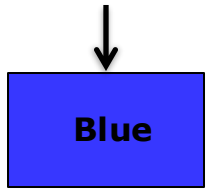


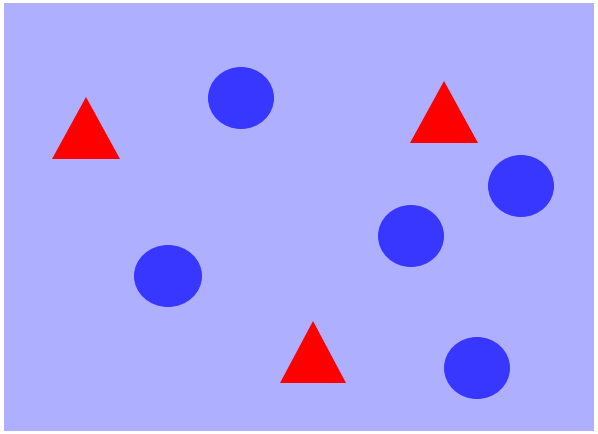
- Greedy:
  - Given budget of k internal nodes in decision tree:
  - Start with root that is a leaf.
  - Repeat k times:
    - Try all leaves and replace by best decision stump.
    - Use the best of these choices (best accurracy).

Small Decision Trees

- Repeat k times:
  - Try all leaves and replace by best decision stump.
  - Use the best of these choices.

k=2 example

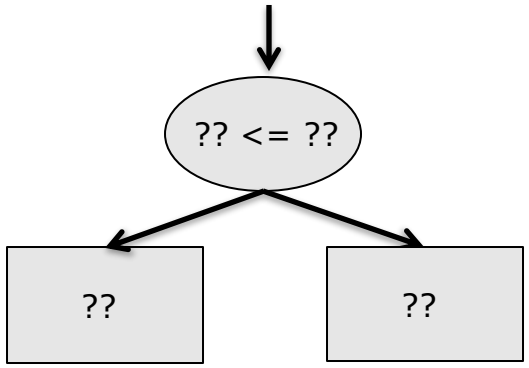


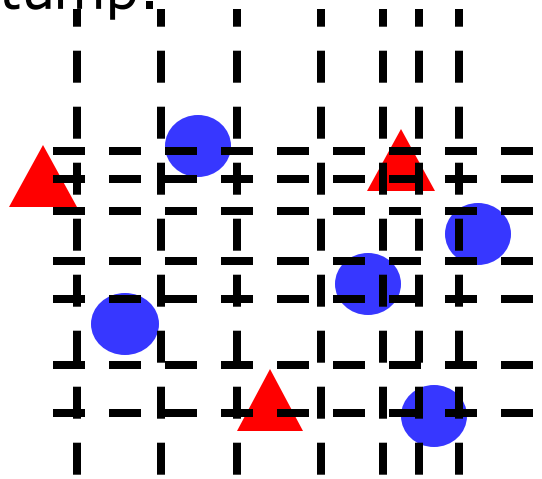


Small Decision Trees

- Repeat k times:
  - Try all leaves and replace by best decision stump.
  - Use the best of these choices.

k=2 example

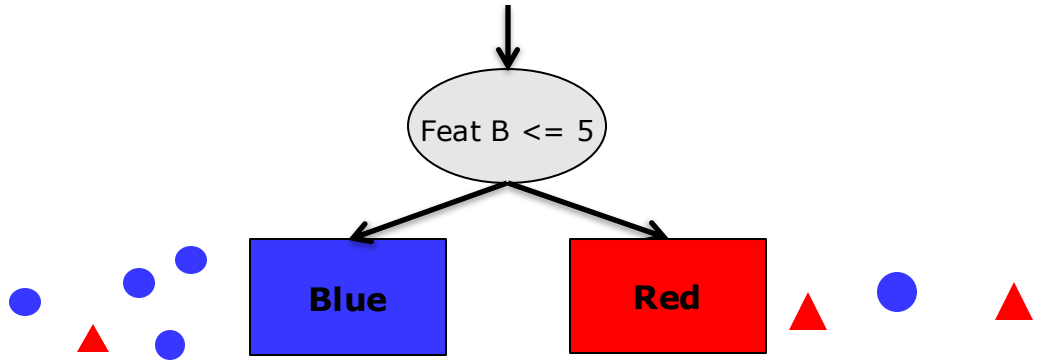


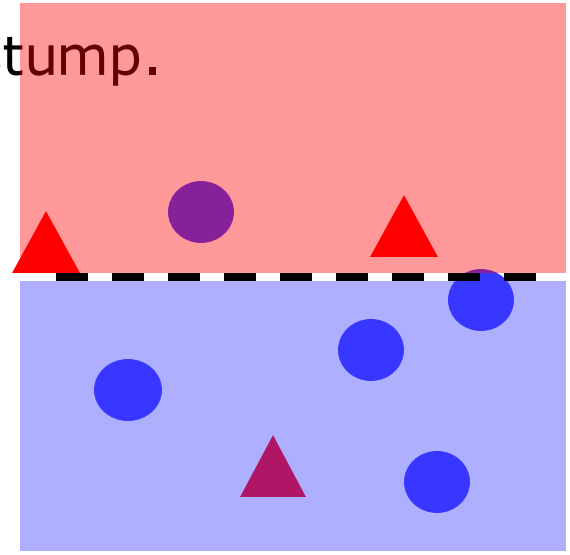


Small Decision Trees

- Repeat k times:
  - Try all leaves and replace by best decision stump.
  - Use the best of these choices.

k=2 example



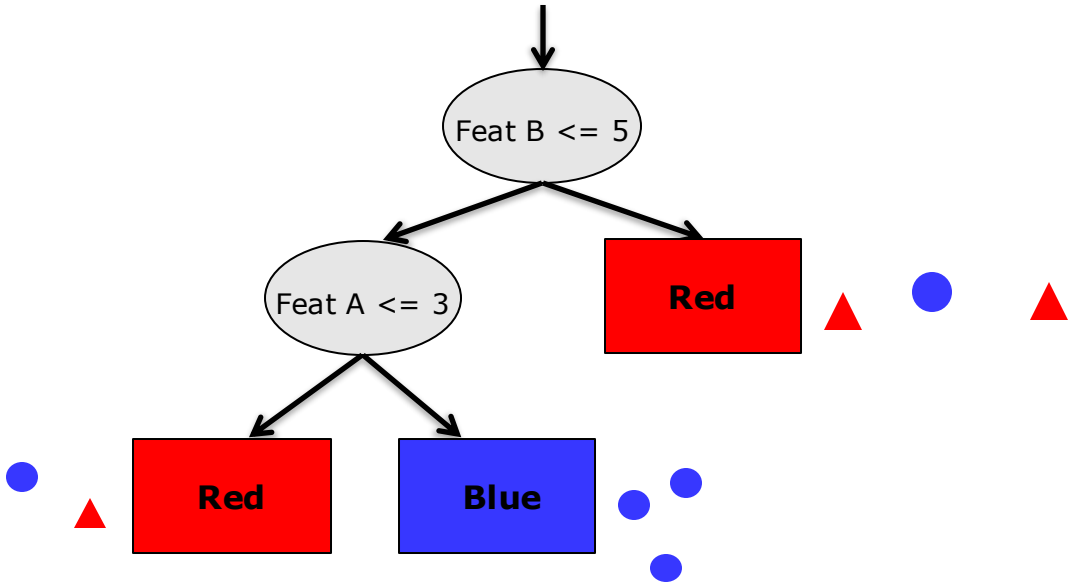


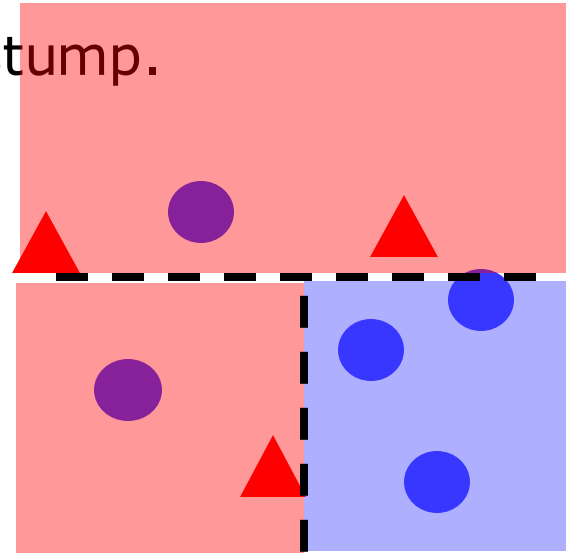
Accuracy: 6/8

Small Decision Trees

- Repeat k times:
  - Try all leaves and replace by best decision stump.
  - Use the best of these choices.

k=2 example



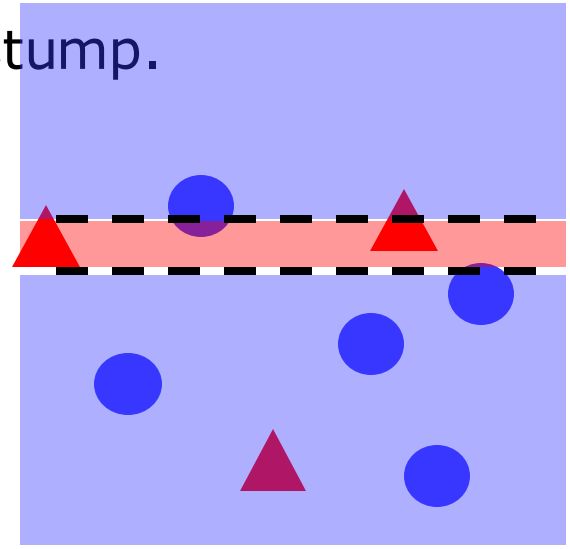


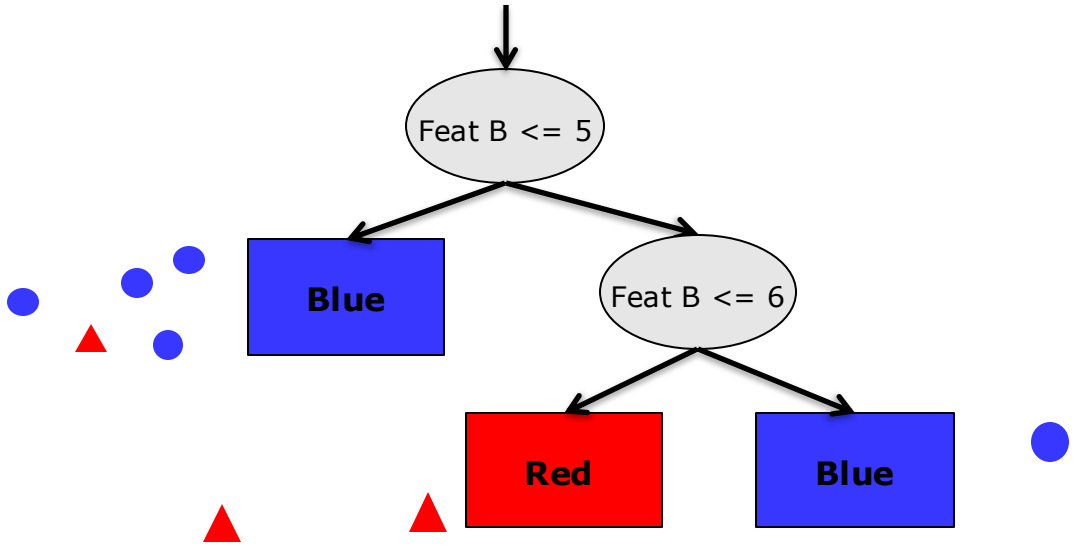
Accuracy: 6/8

Small Decision Trees

- Repeat k times:
  - Try all leaves and replace by best decision stump.
  - Use the best of these choices.

k=2 example



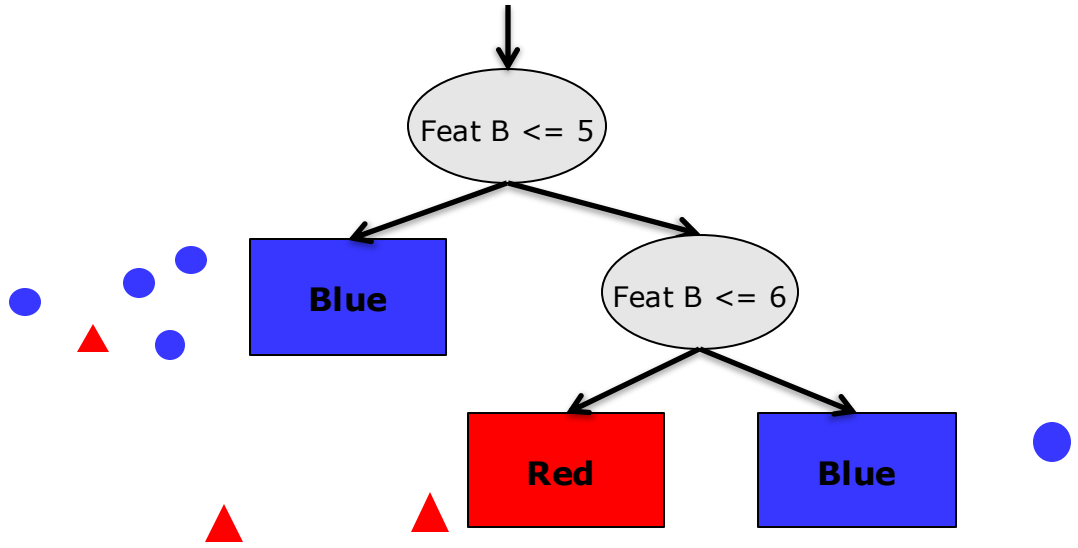


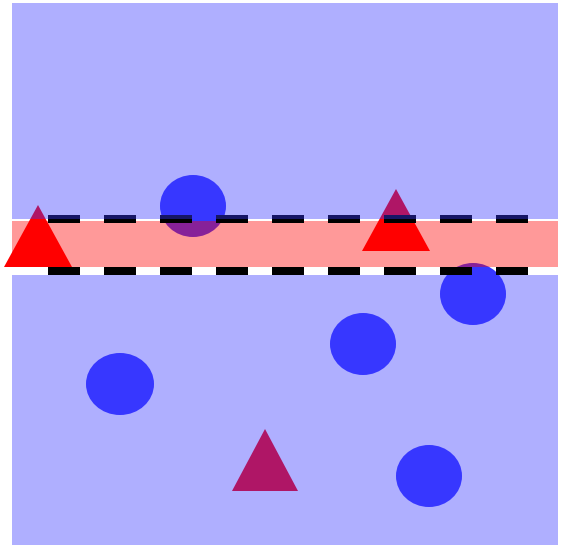
Accuracy: 7/8

Small Decision Trees

- Final Tree:

k=2 example





Accuracy: 7/8

Small Decision Trees

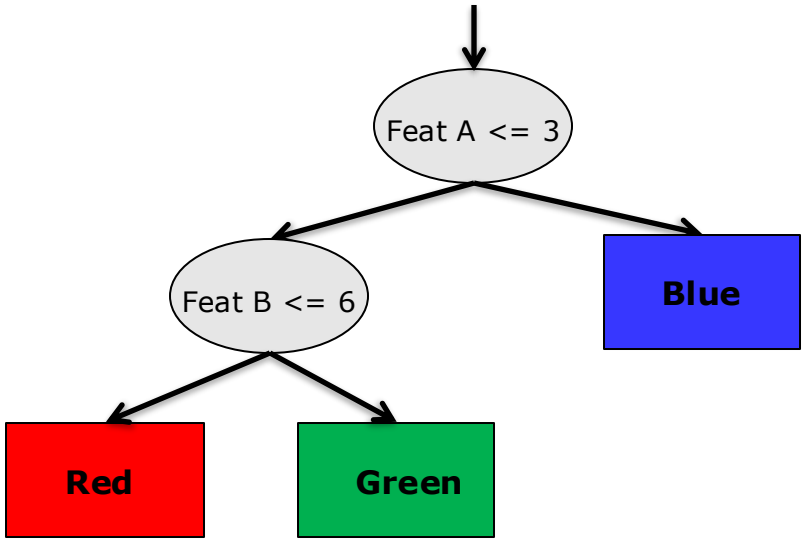
- Greedy:
  - Given budget of k internal nodes in decision tree:
  - Start with root that is a leaf.

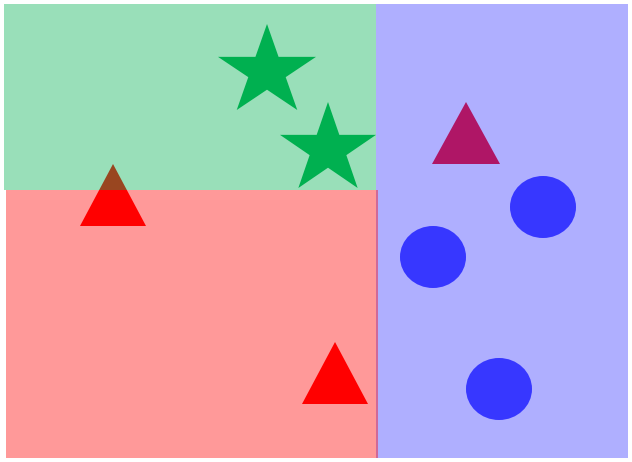
  - Repeat k times:
    - Try all leaves and replace by best decision stump.
    - Use the best of these choices.

- Other approach:
  - Given budget of depth d.
  - Start with root that is a leaf.
  - Repeat d times:
    - Replace all leaves by best decision stump.

More Classes/Labels/Categories

- Repeat k times:
  - Try all leaves and replace by best decision stump.
  - Use the best of these choices.





- Best leaf: Still output most frequent label that reach leaf.

Small Decision Trees

- Growing Small Trees:
  - NP-hard to find best.
  - Therefore many different heuristics.
  - Popular implementations vary in how they find trees.

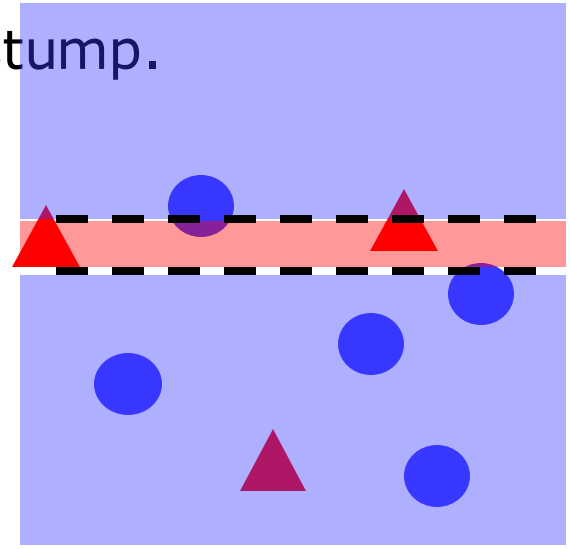
- In next part of this lecture:
  - Another heuristic for growing small classification trees (based on entropy/information gain).
  - Other error measures / loss functions.
  - Regression trees (predict value instead of label).

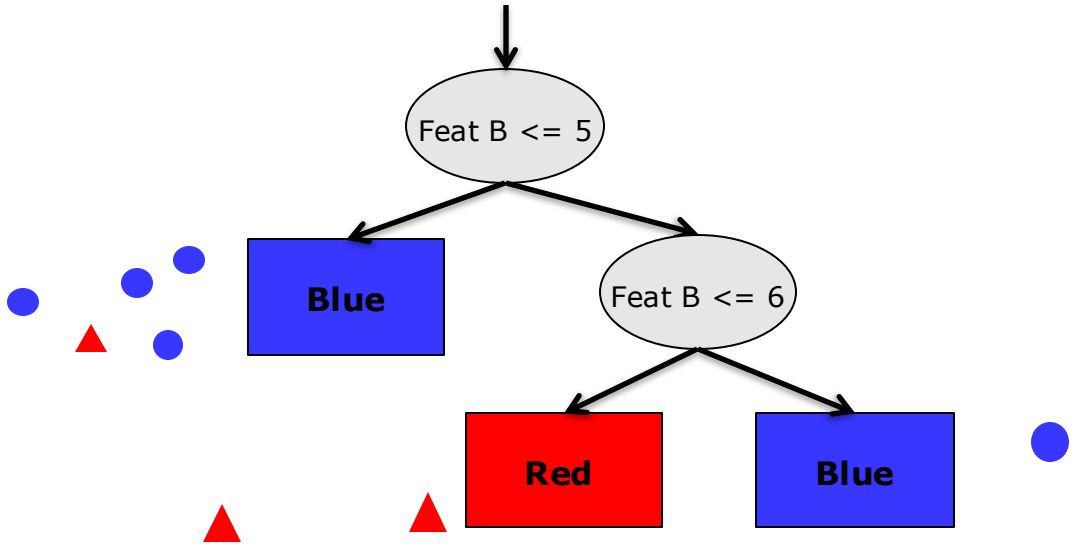
Decision Trees

Small Decision Trees

- Repeat k times:
  - Try all leaves and replace by best decision stump.
  - Use the best of these choices.

k=2 example





Accuracy: 7/8

Entropy

- Other measure of best?
  - Popular heuristic: Base it on entropy.

- If have n points with labels 1,…,k in a set S, then:
  - Let n_i be number of points with label i.
  - Let $p_i = n_i/n$ be fraction of points with label i.

  - Entropy: $\mathrm{H}(\mathrm{S}) = -\sum_{i=1}^{k} p_i \log_2 p_i$ Note: $0 \log_2 0 = 0$

- In information theory, (Shannon) entropy is a measure of uncertainty in a distribution.
- High entropy means high uncertainty.

Entropy

- Entropy
  - $\mathrm{H}(\mathrm{S}) = -\sum_{i=1}^{k} p_i \log_2 p_i$
- Intuitively: H(S) describes the expected number of bits it takes to describe the label of a random element drawn from S.
- All elements in S are equal:
  - $p_i=1$, $p_j=0$ for $i \neq j$.
  - $\mathrm{H}(\mathrm{S}) = -(1 \log_2 1 + (\mathrm{k}-1)0 \log_2 0) = 0$.
- Equally many of each in S:
  - $p_i=1/k$.
  - $\mathrm{H}(\mathrm{S}) = -k(\frac{1}{k} \log_2 \frac{1}{k}) = \log_2 k$

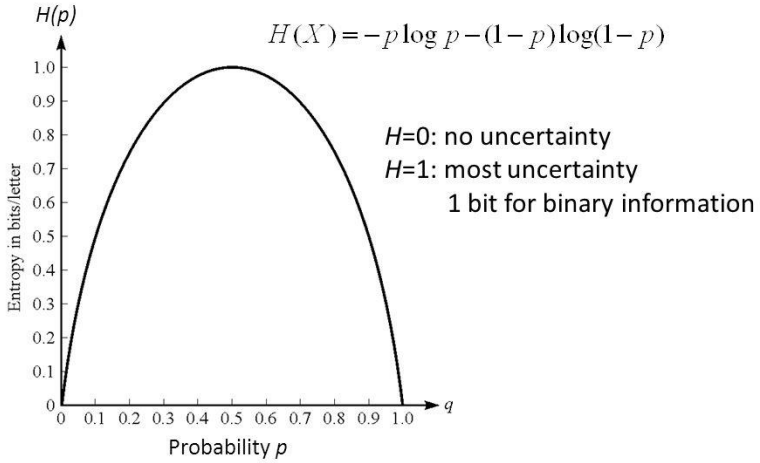
Entropy

- Entropy
  - $\mathrm{H(S)} = -\sum_{i=1}^{k} p_i \log_2 p_i$

- Intuitively: H(S) describes the expected number of bits it takes to describe the label of a random element drawn from S.

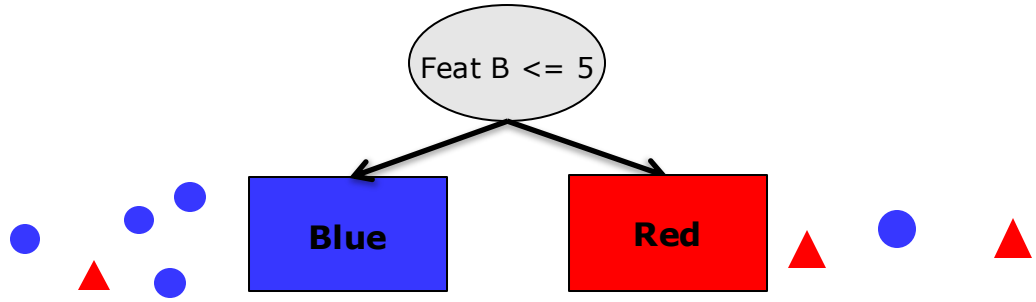
- For 2 elements:

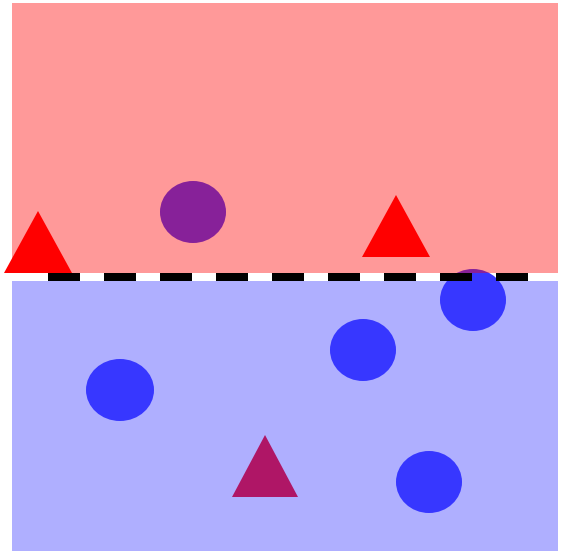
Binary entropy function



Entropy

- Entropy in a decision tree:

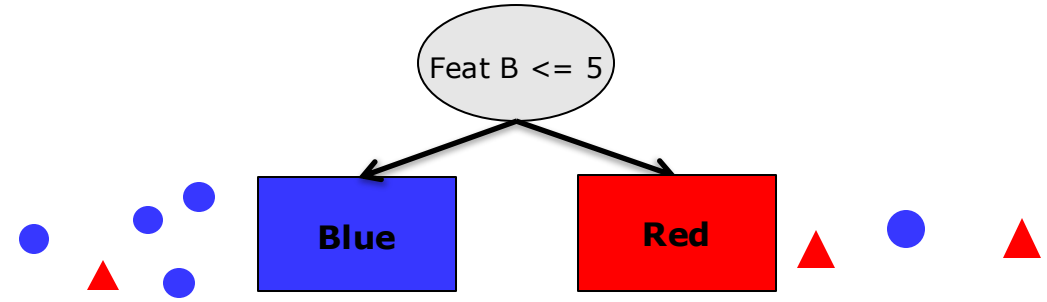


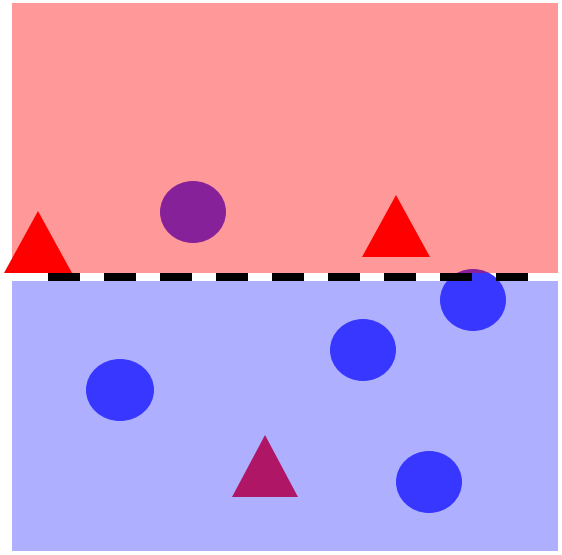


- For each leaf v, let S_v be subset of the n training points ending there.
- Entropy of tree: $\sum_v \frac{|S_v|}{n} H(S_v)$
  - Sum over all leaves, entropy in leaf, weighed by fraction of training points ending there.

Entropy

- Entropy of tree: $\sum_v \frac{|S_v|}{n} H(S_v)$





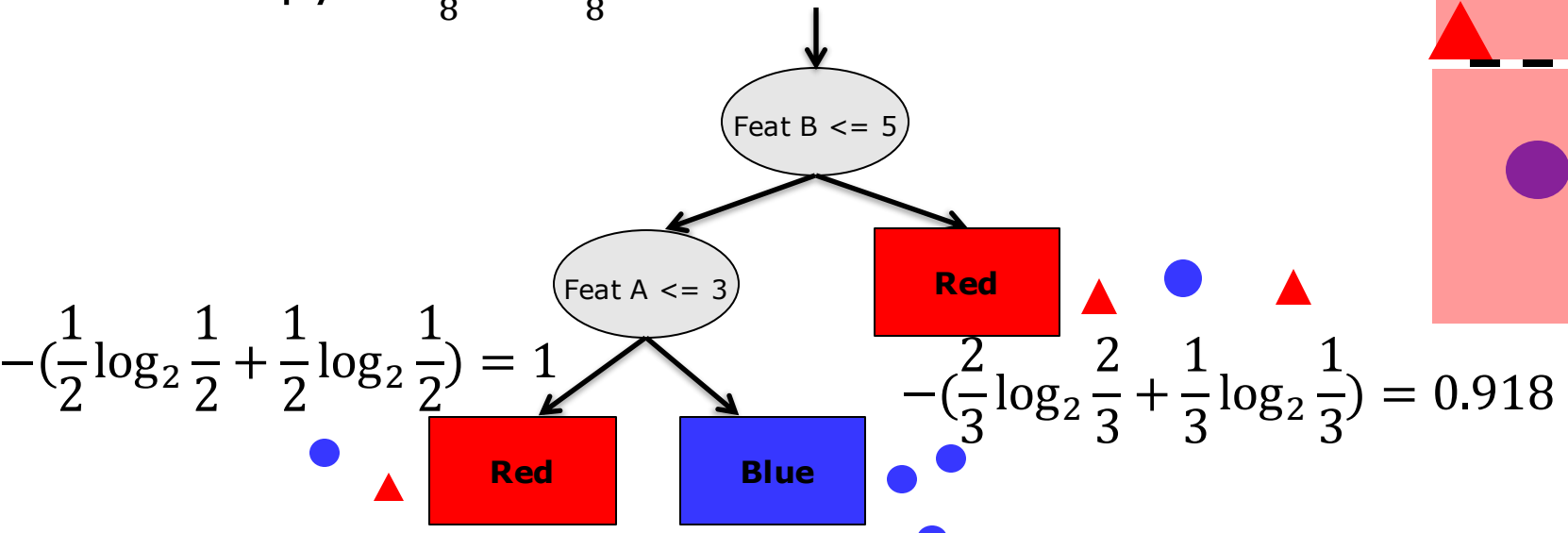
$$-\left(\frac{1}{5}\log_2\frac{1}{5}+\frac{4}{5}\log_2\frac{4}{5}\right)=0.722 \qquad -\left(\frac{2}{3}\log_2\frac{2}{3}+\frac{1}{3}\log_2\frac{1}{3}\right)=0.918$$

Entropy is: $\frac{5}{8}\cdot 0.722+\frac{3}{8}\cdot 0.918=0.796$

Entropy

- Entropy of tree: $\sum_v \frac{|S_v|}{n} H(S_v)$

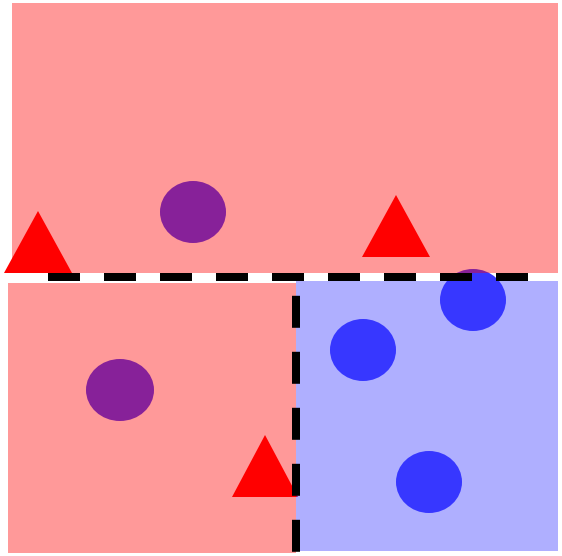
Entropy is: $\frac{2}{8} \cdot 1 + \frac{3}{8} \cdot 0.918 = 0.594$



$-(\frac{1}{2}\log_2\frac{1}{2} + \frac{1}{2}\log_2\frac{1}{2}) = 1$

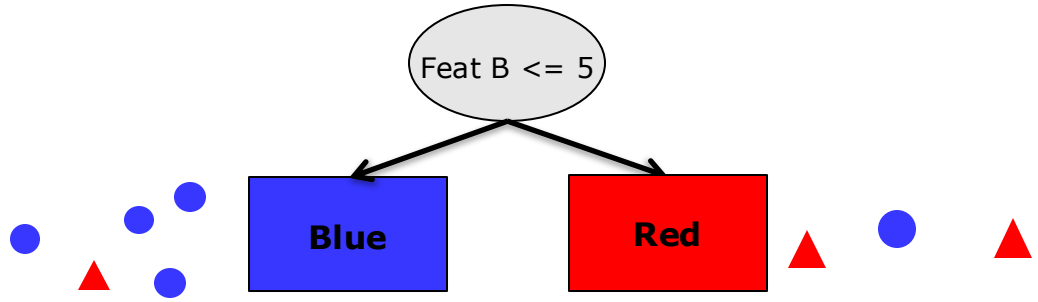
$-(\frac{2}{3}\log_2\frac{2}{3} + \frac{1}{3}\log_2\frac{1}{3}) = 0.918$

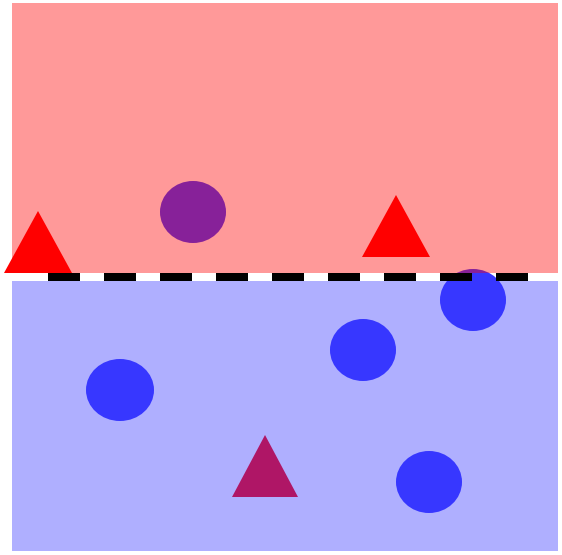
$-(1\log_2 1 + 0\log_2 0) = 0$



Entropy

- Entropy in a decision tree:





- Entropy of tree: $\sum_v \frac{|S_v|}{n} H(S_v)$
- Intuition: Amount of uncertainty/bits needed to describe the label of a random training point if told which leaf it ends in.

Small Decision Trees

- Greedy:
  - Given budget of k internal nodes in decision tree:
  - Start with root that is a leaf.

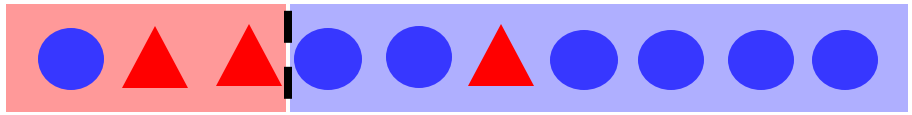
  - Repeat k times:
    - Try all leaves v, all features f and all split values x.
    - For each choice, compute reduction in tree entropy if substitute v with a decision stump splitting using feature f at value x.
    - Use the leaf replacement that reduces entropy the most.

Entropy

- May give different trees than using accurracy.
- 1-dimensional example, i.e. one feature.
  - Entropy before splitting: $-(\frac{3}{10}\log_2\frac{3}{10}+\frac{7}{10}\log_2\frac{7}{10})=0.881$
  - Best split using accurracy:

$$-(\frac{1}{7}\log_2\frac{1}{7}+\frac{6}{7}\log_2\frac{6}{7})=0.592$$

Accuracy: 8/10



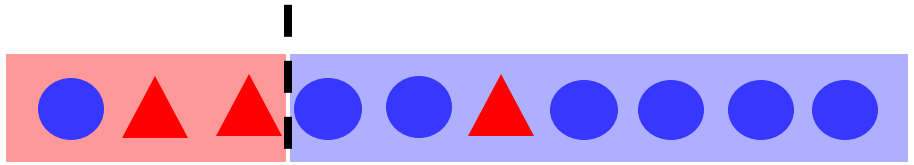
$$-(\frac{2}{3}\log_2\frac{2}{3}+\frac{1}{3}\log_2\frac{1}{3})=0.918$$

  - Entropy after: $\frac{3}{10}\cdot 0.918+\frac{7}{10}\cdot 0.592=0.690$

Entropy

- May give different trees than using accurracy.
- 1-dimensional example, i.e. one feature.
  - Entropy before splitting: $-(\frac{3}{10}\log_2\frac{3}{10}+\frac{7}{10}\log_2\frac{7}{10}) = 0.881$
  - Best split using accurracy:

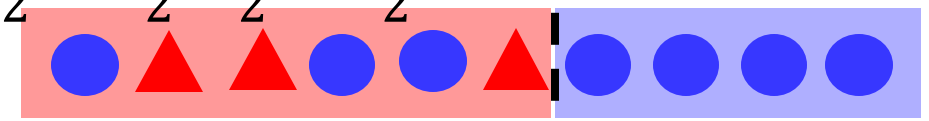
Accuracy: 8/10



  - Entropy after: $\frac{3}{10}\cdot 0.918 + \frac{7}{10}\cdot 0.592 = 0.690$

$-(\frac{1}{2}\log_2\frac{1}{2}+\frac{1}{2}\log_2\frac{1}{2}) = 1$ 0

Accuracy: 7/10



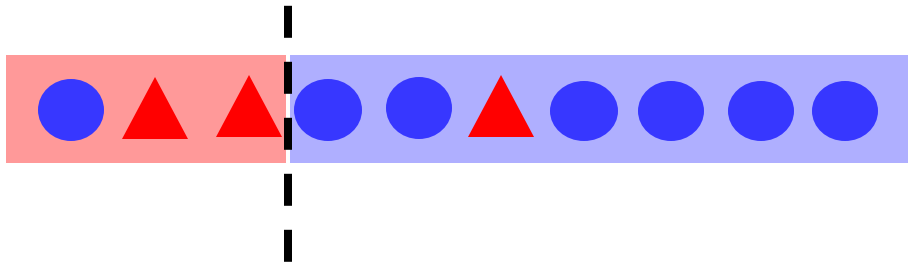
  - Entropy after: $\frac{6}{10}\cdot 1 + \frac{4}{10}\cdot 0 = 0.600$

Larger entropy reduction.

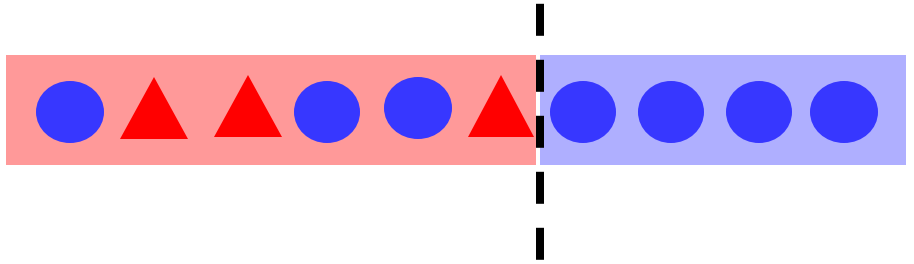
Entropy

- May give different trees than using accurracy.
- 1-dimensional example, i.e. one feature.
  - Entropy prefers ”pure” splits over accurracy.
  - Seem to work best in practice when constructing decision trees (larger than stumps).

Accuracy Split



Entropy Split



Loss Functions

- We have just seen two different methods/goals to optimize when constructing tree.
- In general, one can try to minimize many different objectives.

- Motivating Example:
  - CIA uses fingerprint scanner at entrace to secure facility.
  - Two classes:
    - +1: Authorized
    - -1: Unauthorized
  - Train model to recognize fingerprints based on data.

  - Observation: Much worse to allow an unauthorized to enter than to deny an authorized.

Loss Functions

- Motivating Example:
  - CIA uses fingerprint scanner at entrace to secure facility.
  - Two classes:
    - +1: Authorized
    - -1: Unauthorized

- Loss Function: Function L(y',y) that determines cost.
  - y' is predicted label.
  - y is true label.

- In example:
  - L(1,1) = L(-1,-1) = 0.
  - L(1,-1) = 1000, L(-1,1) = 1.

Loss Functions

- In Supervised Learning:
  - Minimize the loss L over the training data $(x_1,y_1),\ldots,(x_n,y_n)$.
    - Find hypothesis g minimizing $\frac{1}{n}\sum_{i=1}^{n} L(g(x_i),y_i)$.
    - Hope minimizes same loss on new unknown data.

- Maximizing accurracy:
  - So far, we talked about maximizing accurracy.
  - Same as minimizing the 0-1 Loss:
    - $L(y',y) = 1$ if $y' \neq y$, else $L(y',y)=0$.

- Greedy Decision Tree Construction:
  - In leaves, return label that minimizes loss. Still try all splits.

Loss Functions

- Regression
  - Predicting value instead of class/category.
  - Labels Y=R (real values).



- Predict house prices.

Loss Functions

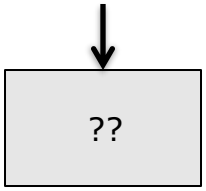
- Least Squares Loss Function
  - In regression, often use least squares loss.
  - $L(y',y) = (y'-y)^2$

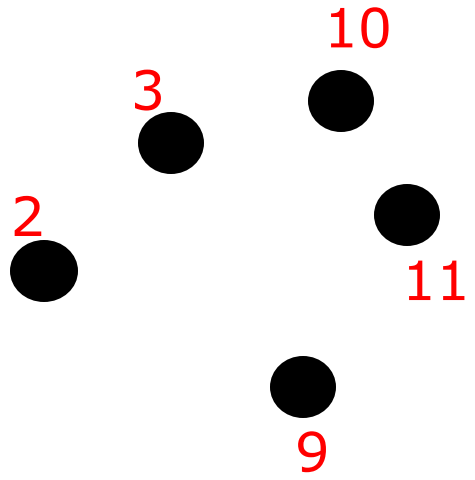


- $L(5.300.000, 5.150.000) = (150.000)^2$

Small Regression Trees

- Greedy Learning Algorithm.
  - Start in simple case:
    - Our tree may only contain 1 node.
    - Root is a leaf.

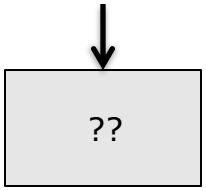


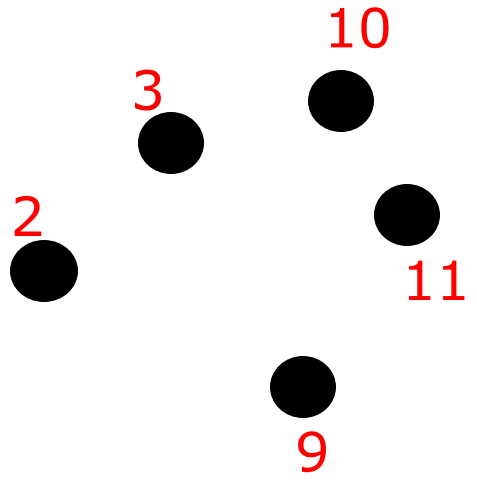


- What is best choice of root?
  - It must return a value x.
  - If return x, then loss becomes:
  - $\frac{1}{n}\sum_{i=1}^{n}(x-y_i)^2$ =((2-x)^2+(3-x)^2+(9-x)^2+(10-x)^2+(11-x)^2)/5.
  - Must choose x to minimize this.

Small Regression Trees

- Greedy Learning Algorithm.
  - Start in simple case:
    - Our tree may only contain 1 node.
    - Root is a leaf.

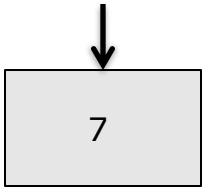


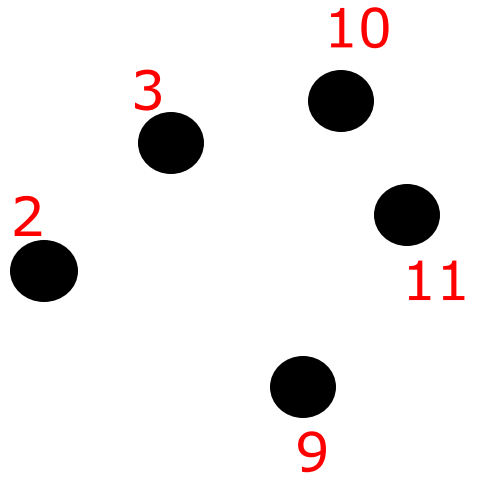


- What is best choice of root?
  - The x that minimizes $\frac{1}{n}\sum_{i=1}^{n}(x-y_i)^2$ is the mean:
    - $x = \frac{1}{n}\sum_{i=1}^{n} y_i$.

Small Regression Trees

- Greedy Learning Algorithm.
  - Start in simple case:
    - Our tree may only contain 1 node.
    - Root is a leaf.





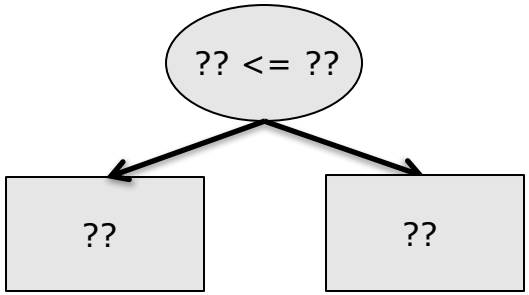
- What is best choice of root?
  - The x that minimizes $\frac{1}{n}\sum_{i=1}^{n}(x-y_i)^2$ is the mean:
    - $x = \frac{1}{n}\sum_{i=1}^{n} y_i$.

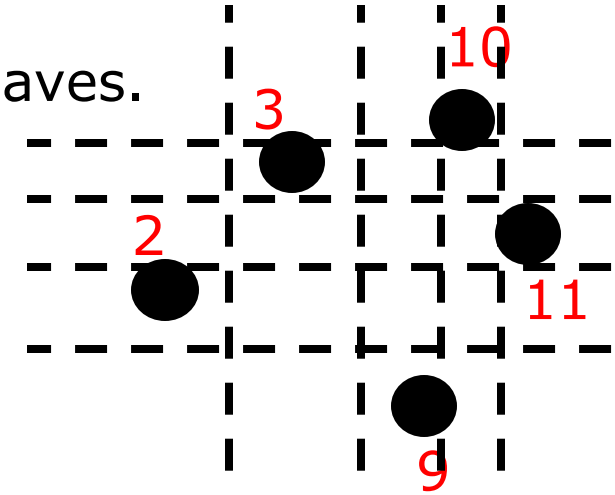
Mean is (2+3+9+10+11)/5=35/5=7

Loss is ((2-7)^2+(3-7)^2+(9-7)^2+(10-7)^2+(11-7)^2)/5=14.

Small Regression Trees

- Greedy Learning Algorithm.
  - Regression Stumps:
    - One root and two children that are leaves.

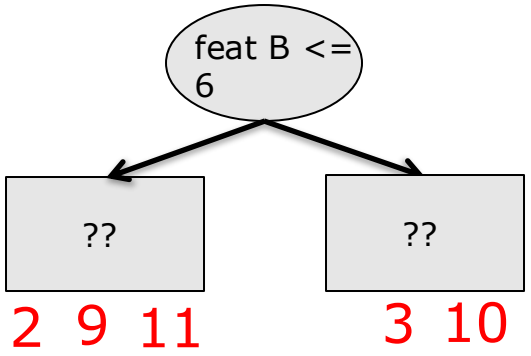


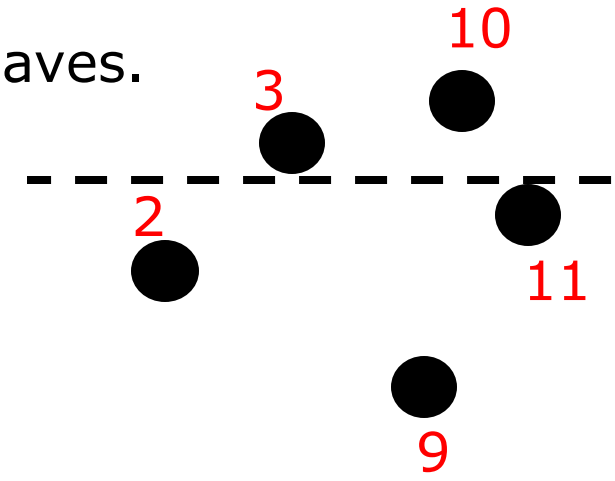


- What is best choice of root?
  - For every feature, there are n-1 relevant splits with n training points.
    - Test all.

Small Regression Trees

- Greedy Learning Algorithm.
  - Regression Stumps:
    - One root and two children that are leaves.

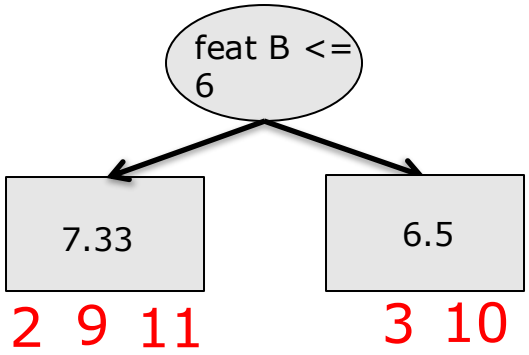


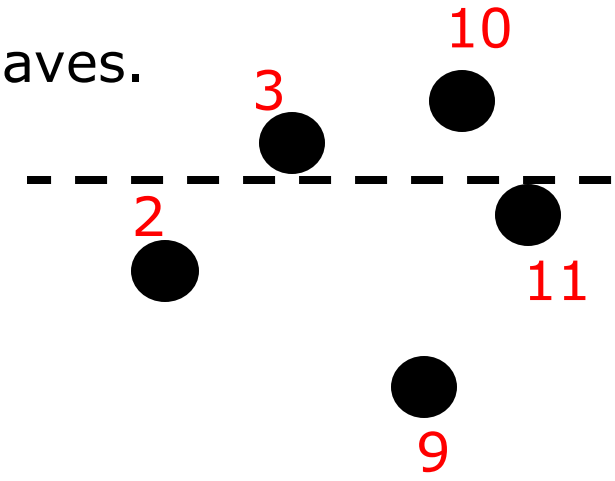


- Test a split:

Small Regression Trees

- Greedy Learning Algorithm.
  - Regression Stumps:
    - One root and two children that are leaves.

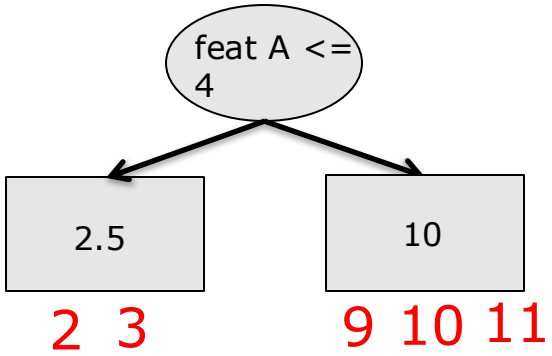


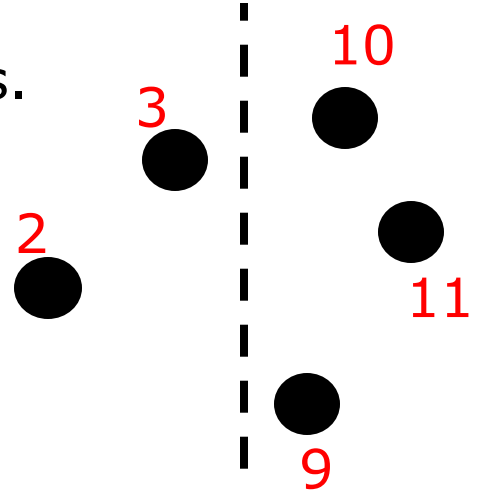


- Test a split:
  - $((2-7.33)^2+(9-7.33)^2+(11-7.33)^2+(3-6.5)^2+(10-6.5)^2)/5$
  - $= 13.833$

Small Regression Trees

- Greedy Learning Algorithm.
  - Regression Stumps:
    - One root and two children that are leaves.





- Test a split:
  - $((2-2.5)^2+(3-2.5)^2+(9-10)^2+(10-10)^2+(11-10)^2)/5$
  - $= 0.5$

- Best split!

Small Regression Trees

- Greedy:
  - Given budget of k internal nodes in regression tree:
  - Start with root that is a leaf.

  - Repeat k times:
    - Try all leaves and replace by best regression stump.
    - Use the best of these choices (smallest loss).

Summary

- Introduced decision trees.
  - New hypothesis set.
- Optimal construction of leaves and stumps.
- Greedy approach for larger trees.
- Entropy-based greedy construction.

- Loss Functions.
  - 0-1 Loss.
  - Least squares.

- Constructing regression trees.
  - Greedy, using least squares loss.